

Database liberi (autore: Vittorio Albertoni)

Introduzione

Se abbiamo una biblioteca, o una cineteca, o una raccolta di dischi, al raggiungimento di un certo numero di elementi finisce, come si suol dire, che perdiamo il conto. Finisce che non ricordiamo più quali brani abbiamo di Brahms piuttosto che di Verdi, quali brani abbiamo diretti da Riccardo Muti, quali brani abbiamo della musica francese del '700, ecc.

Per disporre di queste informazioni, nel momento in cui non possiamo più fare affidamento sulla nostra memoria, dobbiamo creare un archivio che contenga tutti i dati che concorrono a formarle. Per sapere quali brani abbiamo di Brahms, e di quanti altri autori, dobbiamo creare un archivio che contenga autore e titolo del brano. Per sapere quali brani e di quali autori abbiamo diretti da Muti, dobbiamo creare un archivio che contenga autore, titolo del brano e interprete. E così via.

Questo archivio si organizza in una tabella nella quale ciascuna riga è dedicata a un brano e ciascuna colonna è dedicata ai dati che sono interessanti per le informazioni che vogliamo avere. Nel caso ci interessi solo sapere i brani per autore ci basteranno due colonne, una dedicata agli autori, una dedicata ai titoli dei brani. Nel caso ci interessino anche gli interpreti occorreranno tre colonne, una dedicata agli autori, una dedicata ai titoli e una dedicata agli interpreti. E così via.

Nel linguaggio dei database le righe della tabella costruita a questi fini si chiamano **record** e le zone del record corrispondenti alle colonne si chiamano **campi (fields)**.

Visto che si tratta di una tabella, in mancanza d'altro, potremmo costruirla utilizzando un foglio di calcolo. Anzi, i fogli di calcolo che ci sono in circolazione hanno funzioni per l'ordinamento e la ricerca di dati organizzati in tabella. Tuttavia, anche per le applicazioni più elementari, il foglio di calcolo si rivela ben presto inadatto per database di una certa dimensione e con pretese classificatorie di un certo tipo.

Partiamo dalla più banale esigenza: quella di gestire un archivio contenente semplicemente autore e titolo di un brano musicale. Se utilizziamo un foglio di calcolo intestiamo due colonne affiancate, una «autore» e una «titolo» e, riga per riga, inseriamo autore e titolo dei nostri brani musicali. Se abbiamo 18 opere di Ciaikovski dobbiamo ovviamente scrivere 18 titoli, molto probabilmente diversi tra loro, nel campo dedicato al titolo e dobbiamo anche scrivere per 18 volte la stessa cosa nel campo dedicato all'autore, augurandoci di scriverla sempre nello stesso modo: se una volta scriviamo Ciaikovski, un'altra Tchaikovsky, un'altra ancora Chaikovskij vai poi a tirare fuori i titoli delle opere di Ciaikovski che abbiamo nella nostra raccolta di dischi!

Se utilizziamo un altro modello di database in cui facciamo due tabelle, una per gli autori, l'altra per i titoli, entrambe con due colonne, scriveremo una sola volta, badando bene di scriverlo giusto, il nome dell'autore nella tabella degli autori, nella quale gli verrà a corrispondere, nella colonna a fianco, un numero identificativo assegnato solo a lui e scriveremo i nostri 18 titoli nella tabella dei titoli, nella quale, nella colonna a fianco indicheremo il numero dell'autore: il numero che compare in entrambe le tabelle costituisce la relazione tra le due tabelle e il database che funziona in questo modo si chiama database relazionale.

La banalità del contenuto dell'informazione da elaborare e la scarsità dei dati necessari per farlo, in questo esempio, non rende giustizia della enorme efficienza del database relazionale rispetto al database monotabellare gestibile con un foglio di calcolo.

La finalità dell'esempio era tuttavia quella di far capire la differenza che c'è tra le due impostazioni. Per la comprensione dell'efficienza rimando a quanto si scoprirà leggendo il contenuto di questo manuale nel quale tratto tre perle del software libero per gestire database relazionali.

Indice

I	Base di LibreOffice	3
1	Creazione del database	3
2	Alimentazione del database	6
2.1	lavorando sulla tabella	6
2.2	attraverso formulario	7
3	Ricerche sul database	10
3.1	Creazione di ricerca in vista struttura	10
3.2	Creazione di ricerca con la procedura guidata	12
3.3	Creazione di ricerca con il linguaggio SQL	13
4	Rapporti	14
II	MariaDB	15
1	Caratteristiche	15
2	MariaDB a riga di comando	16
2.1	Installazione	16
2.2	Uso di MariaDB a riga di comando	17
3	Uso di MariaDB con LibreOffice Base	23
4	Collegamenti attraverso i principali linguaggi di programmazione.	25
5	MariaDB su server web	26
5.1	Installazione dell'occorrente	27
5.2	phpMyAdmin	28
5.3	MariaDB a riga di comando da XAMPP	31
5.4	MariaDB da XAMPP con LibreOffice Base	31
III	Sqlite	32
1	Installazione	32
2	Uso di SQLite a riga di comando	33
3	Interfacce grafiche per SQLite	39
4	Uso di SQLite con LibreOffice Base	41
5	Collegamenti attraverso i principali linguaggi di programmazione.	44
6	Conversione di database	46
6.1	da SQLite a MariaDB	47
6.2	da MySQL a SQLite	47

Parte I

Base di LibreOffice

Base, componente del software di produttività personale LibreOffice, è nato nel 2011 da un fork di OpenOffice Base, grazie al team di sviluppo della The Document Foundation, organizzazione no profit per lo sviluppo di software libero.

Possiamo procurarci tutta la suite LibreOffice all'indirizzo

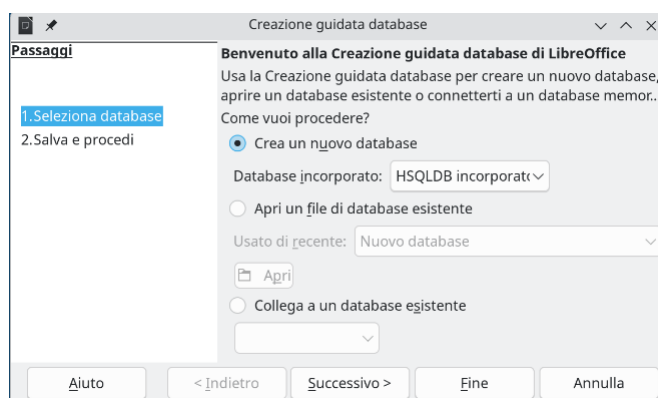
<https://it.libreoffice.org/scarica/>

dove troviamo le versioni per Linux, Windows e Mac.

Capita spesso che la suite si trovi preinstallata su Linux, ma senza il modulo Base. In tal caso è possibile installarlo ricorrendo all'installatore di pacchetti della distro.

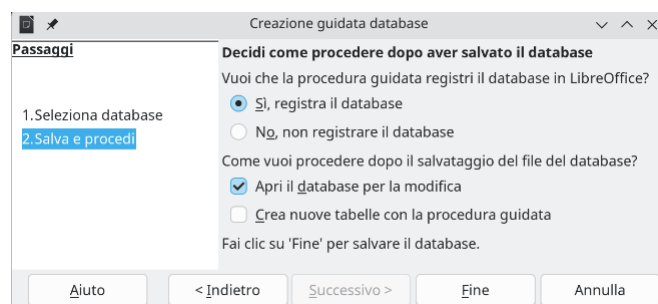
1 Creazione del database

Se lanciamo l'esecuzione del modulo Base di LibreOffice ci si presenta questa finestra



nella quale abbiamo la possibilità di scegliere se vogliamo creare un nuovo database (scelta opzionata per default), se vogliamo aprire un database esistente creato con Base o se vogliamo collegarci ad un altro database.

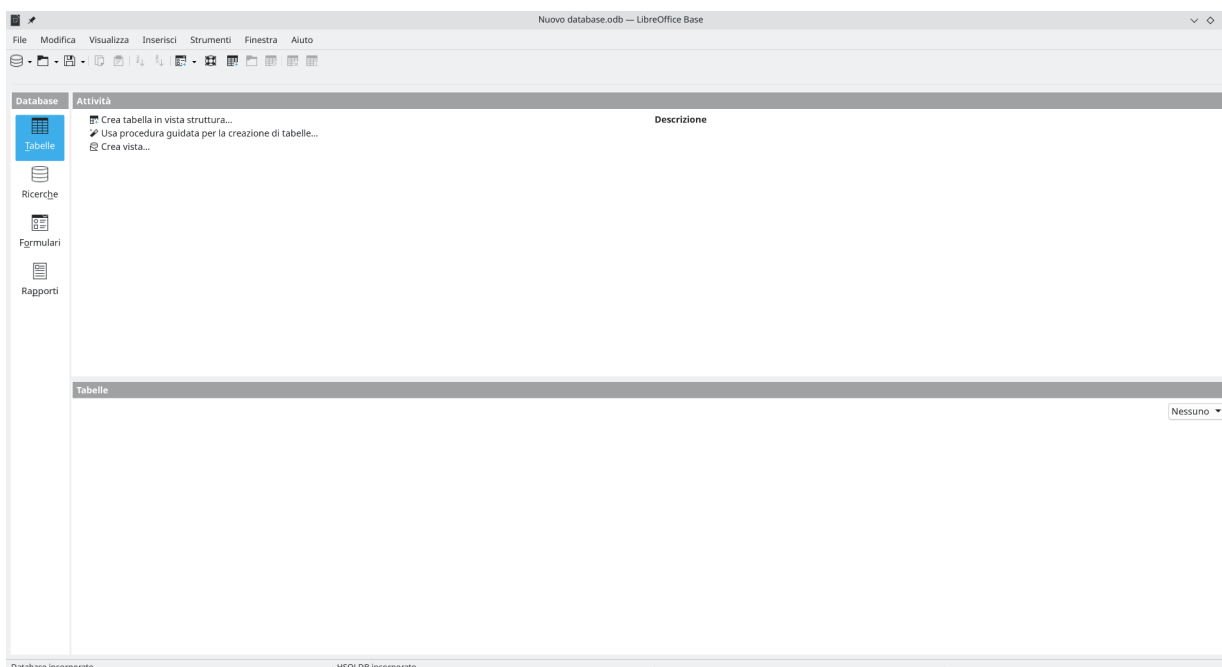
Per creare un nuovo database manteniamo la scelta di default e clicchiamo sul pulsante SUCCESSIVO> aprendo questa nuova finestra



Conserviamo le scelte proposte per default e clicchiamo su FINE.

Si apre così una finestra di dialogo che ci chiede di indicare un nome da assegnare al database e di scegliere il luogo dove salvarlo.

Proseguendo ci troviamo di fronte alla finestra principale del database di LibreOffice



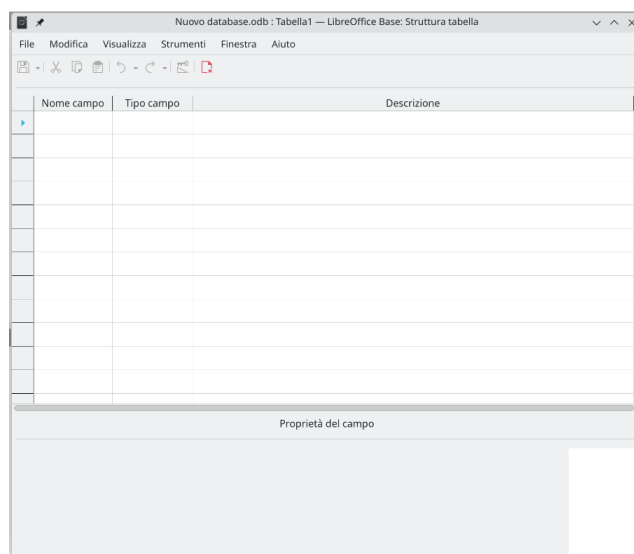
La barra dei menu è autoesplicativa e passando il mouse sulle icone della barra degli strumenti vediamo di cosa si tratta.

Nella barra verticale sulla sinistra possiamo scegliere tra quattro tipi di attività da compiere sul database:

- . TABELLE, per lavorare sulle tabelle del database,
- . RICERCHE, per effettuare ricerche sul database,
- . FORMULARI, per creare o utilizzare finestre formulario per lavorare sul database,
- . RAPPORTI, per creare rapporti su dati selezionati dal database.

Dal momento che dobbiamo creare un nuovo database la prima cosa da fare è creare le tabelle che lo devono comporre e manteniamo pertanto la scelta che compare per default sulle attività relative alle tabelle.

Per non complicarci inutilmente la vita scegliamo l'attività CREA TABELLE IN VISTA STRUTTURATA..., aprendo questa finestra



A questo punto, per procedere, dobbiamo avere le idee chiare sul database che vogliamo creare. Dobbiamo cioè progettare il database.

Affidiamoci ad un semplice esempio semplificato.

Supponiamo di avere una biblioteca di quattro libri e di voler costruire un database per gestirla.

I libri sono

I promessi sposi di Alessandro Manzoni,
Un luogo chiamato libertà di Ken Follett,
I pilastri della terra di Ken Follett,
Furore di John Steinbeck.

Chiediamoci ora a quali esigenze debba far fronte la gestione di questa biblioteca e supponiamo che esse siano:

- . sapere le opere che abbiamo dei vari autori,
- . sapere le opere che abbiamo scritte in certi periodi,
- . sapere gli autori presenti nella nostra biblioteca per Paesi di appartenenza,
- . sapere quali opere abbiamo di autori di un certo Paese.

Da qui deriviamo che i dati che dovremo inserire nel database sono: nomi degli autori, titoli delle opere, date di nascita e di morte degli autori, Paesi di appartenenza degli autori.

Il nostro database relazionale può pertanto essere impostato su tre tabelle: una per gli autori, contenente il loro nome, l'anno della nascita e l'anno della morte, una per i Paesi di appartenenza e una per i titoli delle opere. La tabella degli autori si deve relazionare con quella dei Paesi e con quella dei titoli delle opere e quest'ultima basta che si relazioni con quella degli autori, attraverso la quale finisce per relazionarsi anche con la tabella dei Paesi.

Ogni tabella, oltre alle colonne corrispondenti ai vari dati da inserire, deve contenere una colonna dedicata alla così detta chiave primaria, un numero progressivo ad inserimento automatico che identifica ciascuna riga della tabella (record).

Inoltre occorre prevedere in ciascuna tabella le colonne necessarie per instaurare le relazioni.

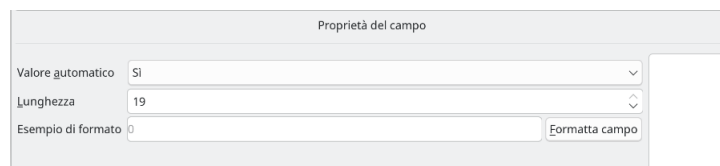
Per impostare le nostre tabelle ricorriamo alla finestra, riprodotta nella pagina precedente, che si apre avendo scelto l'attività CREA TABELLE IN VISTA STRUTTURA...

Per la tabella degli autori inseriamo questi elementi

Nome campo	Tipo campo
idAutore	Intero lungo [BIGINT]
nome	Testo [VARCHAR]
nascita	Intero [INTEGER]
morte	Intero [INTEGER]
idPaese	Intero [INTEGER]

Digitato il nome del campo (intestazione della colonna della tabella) nella zona NOME CAMPO, clicchiamo nella zona TIPO CAMPO e scegliamo il tipo del campo dal menu.

Per l'idAutore, campo destinato alla chiave primaria, scegliendo intero lungo praticamente non poniamo limiti alla dimensione del database. Per questo campo è importante ricordare di scegliere l'opzione Sì per VALORE AUTOMATICO nella zona PROPRIETÀ CAMPO che compare in fondo alla finestra di inserimento



Per il campo destinato al nome dell'autore scegliamo il tipo Testo, proposto per default e per gli altri campi, trattandosi di campi destinati a contenere numeri interi, scegliamo il tipo Intero.

Da menu FILE ► SALVA CON NOME salviamo la tabella così impostata dandole il nome autori.

Nello stesso modo impostiamo la tabella paesi

Nome campo	Tipo campo
idPaese	Intero lungo [B
paese	Testo [VARCHAR

e la tabella opere

Nome campo	Tipo campo
idTitolo	Intero lungo [BIG
titolo	Testo [VARCHAR]
idAutore	Intero [INTEGER]

A questo punto abbiamo creato il database per gestire la nostra piccola biblioteca.

Nella zona TABELLE della finestra principale del database aperto sull'attività TABELLE troviamo elencate le tre tabelle appena impostate.

2 Alimentazione del database

L'inserimento dei dati nel database può avvenire in due modi:

- . lavorando sulla tabella,
- . attraverso formulario.

2.1 lavorando sulla tabella

E' la strada basica: si apre la tabella e si scrivono i dati, riga per riga, nelle varie colonne.

La tabella si apre con doppio click sul suo nome nella zona TABELLE della finestra principale del database aperto sull'attività TABELLE.

In questo modo, a dati inseriti, le nostre tabelle si presentano così:

- . tabella autori

idAutore	nome	nascita	morte	idPaese
0	Manzoni, Alessandro	1785	1873	0
1	Follett, Ken	1949		1
2	Steinbeck, John	1902	1968	2
<Campo auto				

- . tabella opere

idTitolo	titolo	idAutore
0	I promessi sposi	0
1	Un luogo chiamato libertà	1
2	I pilastri della terra	1
3	Furore	2
<Campo aut		

- . tabella paesi

idPaese	paese
0	Italia
1	Gran Bretagna
2	USA
<Campo aut	

Grossa scomodità per l'inserimento del dato idPaese nella tabella degli autori, che richiede la conoscenza dell'identificativo leggibile dalla tabella dei paesi e del dato idAutore nella tabella delle opere, che richiede la conoscenza dell'identificativo dalla tabella degli autori.

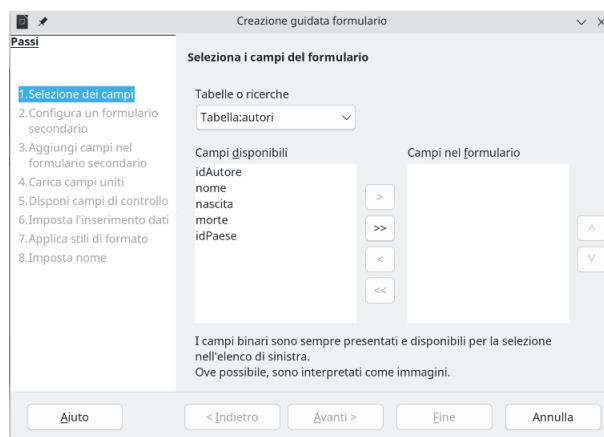
2.2 attraverso formulario

La scomodità cui si è accennato nel finale del precedente paragrafo si supera se inseriamo i dati utilizzando un formulario appositamente costruito.

Visto che il problema non si pone per la tabella paesi, per la quale possiamo benissimo procedere lavorando sulla tabella, vediamo come predisporre due formulari, per la tabella autori e per la tabella opere.

Dalla finestra principale del database clicchiamo su FORMULARI nella barra verticale di sinistra e nella zona delle attività clicchiamo su USA LA PROCEDURA GUIDATA PER LA CREAZIONE DI FORMULARI...

Cominciamo con il formulario per l'inserimento di autori. Nella finestra di dialogo che si è aperta, agendo nella finestrella TABELLE O RICERCHE agiamo in modo che sia selezionata la TABELLA:AUTORI



Tralasciando, tra i CAMPI DISPONIBILI, il campo idAutore, che si alimenta automaticamente e pertanto non occorre inserirlo nel formulario, spostiamo tutti gli altri campi disponibili nella finestrella a destra, dei CAMPI NEL FORMULARIO: lo spostamento avviene selezionando il campo da spostare e cliccando sul pulsante >.

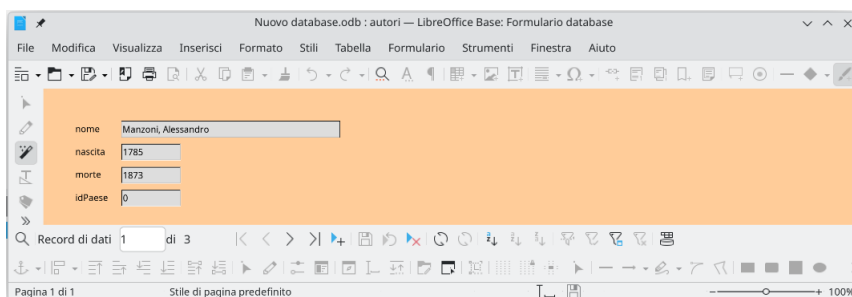
Clicchiamo su AVANTI, accettiamo le proposte di default e clicchiamo nuovamente su AVANTI.

Nella nuova finestra di dialogo, nella zona DISPOSIZIONE DEL FORMULARIO PRINCIPALE, scegliamo questo formato, che mi sembra il migliore



Clicchiamo altre tre volte su AVANTI, accettando le proposte di default e nell'ultima finestra di dialogo, dato un nome al formulario (autori) scrivendolo nella finestrella relativa, clicchiamo su FINE.

Abbiamo così creato questo formulario



Il suo nome compare nella finestra principale aperta sull'attività FORMULARI, nella zona FORMULARI. e da lì lo possiamo aprire con doppio click.

Esso è aperto sul primo record del database e agendo sui pulsanti



possiamo scorrere tra i record.

In particolare l'ultimo pulsante a destra serve per posizionarci dove inserire un nuovo record compilando le varie finestrelle dei campi.

Non ci resta che abilitare la finestrella relativa al campo idPaese a farci scegliere il dato da inserire proponendoci un elenco in chiaro dei Paesi.

Per arrivare a questo dobbiamo innanzi tutto creare una ricerca che produca una tabella dei Paesi, con il nome del Paese in chiaro nella prima colonna e l'idPaese nella seconda.

Andiamo nella scheda di attività Ricerche della finestra principale cliccando sulla relativa voce nella barra verticale di sinistra e scegliamo l'attività CREA RICERCA IN VISTA STRUTTURA.

Nella finestrella di scelta che ci viene proposta scegliamo l'opzione TABELLE (proposta per default) e scegliamo, sotto, la tabella paesi e clicchiamo su AGGIUNGI.

Nella parte inferiore della finestra di lavoro sottostante alla finestrella di scelta agiamo in modo che la ricerca sia così creata

Campo	paese	idPaese
Alias		
Tabella	paesi	paesi
Ordine	crescente	
Visibile	☒	☒

e salviamo la ricerca da menu FILE ► SALVA CON NOME nominandola _paesi (l'underscore iniziale serve a qualificare questa ricerca come ricerca funzionale, per distinguerla dalle ricerche vere e proprie).

Ora dobbiamo modificare la finestrella del nostro formulario per l'inserimento della idPaese, in modo che ci venga proposto l'elenco dei Paesi tra cui scegliere.

Dalla finestra principale in attività FORMULARI clicchiamo destro sul nome del formulario autori che compare nell'elenco dei formulari e scegliamo MODIFICA...

Con premuto il tasto CTRL clicchiamo sulla finestrella per l'inserimento della idPaese, in modo che sia selezionata solo quella.

Clicchiamo destro sulla zona selezionata e scegliamo la voce SOSTITUISCI CON: dal menu che compare e poi scegliamo CASELLA DI RIEPILOGO dal successivo menu.

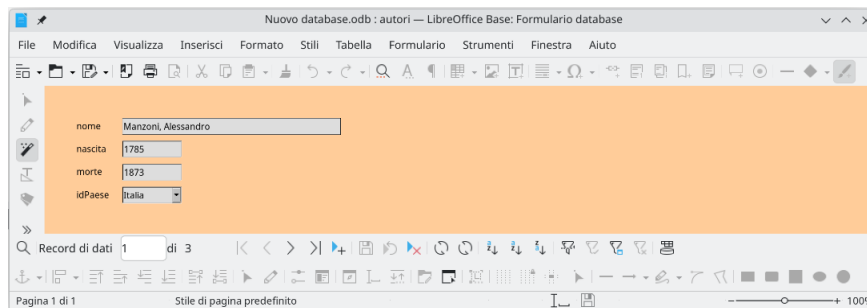
Ora clicchiamo destro sulla solita finestrella selezionata e scegliamo la voce PROPRIETÀ DEL CONTROLLO.

Apriamo la scheda GENERALE della finestra di dialogo che compare e scorriamo fino a trovare la voce APRIBILE, in corrispondenza alla quale scegliamo SI. In questo modo stabiliamo che la casella si possa aprire in un menu a tendina cliccando sul pulsantino che essa conterrà in fondo a destra. Nella riga successiva, NUMERO DELLE RIGHE, possiamo indicare il numero delle righe che debba contenere il menu a tendina alla prima apertura, fermo restando che, se le righe sono di più, il menu conterrà una barra di scorrimento per visualizzare anche le altre.

Utilizzando la scheda DATI scegliamo il dato che dovrà riguardare la casella, nel nostro caso scegliamo idPaese nel menu a tendina che si apre cliccando sul pulsantino in fondo a destra della finestrella relativa al CAMPO DI DATI.

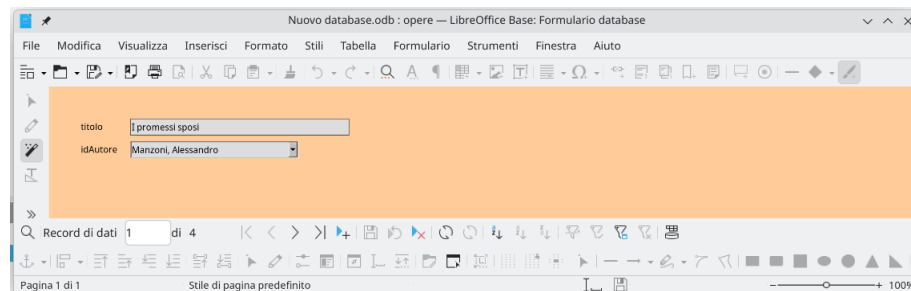
Nella riga TIPO DEL CONTENUTO DELLA LISTA scegliamo Ricerca dal menu a tendina, nella riga CONTENUTO ELENCO scegliamo la ricerca che avevamo creato a questo scopo, che avevamo chiamato _paesi e lasciamo il valore 1 proposto per il CAMPO COLLEGATO.

Salviamo il nostro formulario con il nome autori. Esso si presenta ora in questo modo



Si nota che la finestrella destinata a idPaese contiene in chiaro il nome del Paese, non più, come avveniva prima, il codice del Paese e che sulla destra contiene un pulsantino cliccando sul quale apriamo l'elenco in chiaro dei Paesi: cliccando sul Paese di cui si vuole inserire l'idPaese viene inserito nella tabella del database il relativo numero identificativo.

Con lo stesso procedimento, mutatis mutandis, creiamo il formulario per inserire le opere, che, se lavoriamo bene, si presenterà così



Per allargare la finestrella destinata a contenere il nome dell'autore basta aprire il formulario per la modifica (click destro sul suo nome e scelta relativa alla modifica), selezionare la sola finestrella da allargare (click sulla finestrella con premuto il tasto CTRL) e allargarla agendo con il mouse sulla maniglia di destra.

Nel frattempo la nostra biblioteca si è arricchita di altri tre volumi:

- . Il codice Rebecca di Ken Follett,
- . Caos calmo di Sandro Veronesi (nato nel 1959 e vivente, italiano),
- . Guerra e pace di Lev Tolstoj (nato nel 1828 e morto nel 1919, russo).

Aggiorniamo il database innanzi tutto aggiungendo il Paese Russia nella tabella dei Paesi, senza bisogno di formulario. Il Paese Italia è già presente per Alessandro Manzoni e il Paese di Follett è già presente per lo stesso Follett.

Poi, utilizzando il formulario degli autori, dopo esserci posizionati sul primo record disponibile cliccando sull'ultimo pulsante a destra della zona



inseriamo i due nuovi autori, Veronesi e Tolstoj, con i relativi dati nelle finestrelle vuote, avvalendoci, per l'idPaese, del menu apribile cliccando sulla destra nella relativa finestrella. Compilato il formulario per un autore si passa al successivo premendo il tasto di tabulazione. L'autore Follett è già presente.

Infine, utilizzando il formulario delle opere, inseriamo i tre titoli, avvalendoci, per l'idAutore, del menu apribile cliccando sulla destra nella relativa finestrella.

3 Ricerche sul database

Se apriamo la finestra principale di Base cliccando sull'icona RICERCHE nella barra verticale sulla sinistra apriamo la finestra delle ricerche.

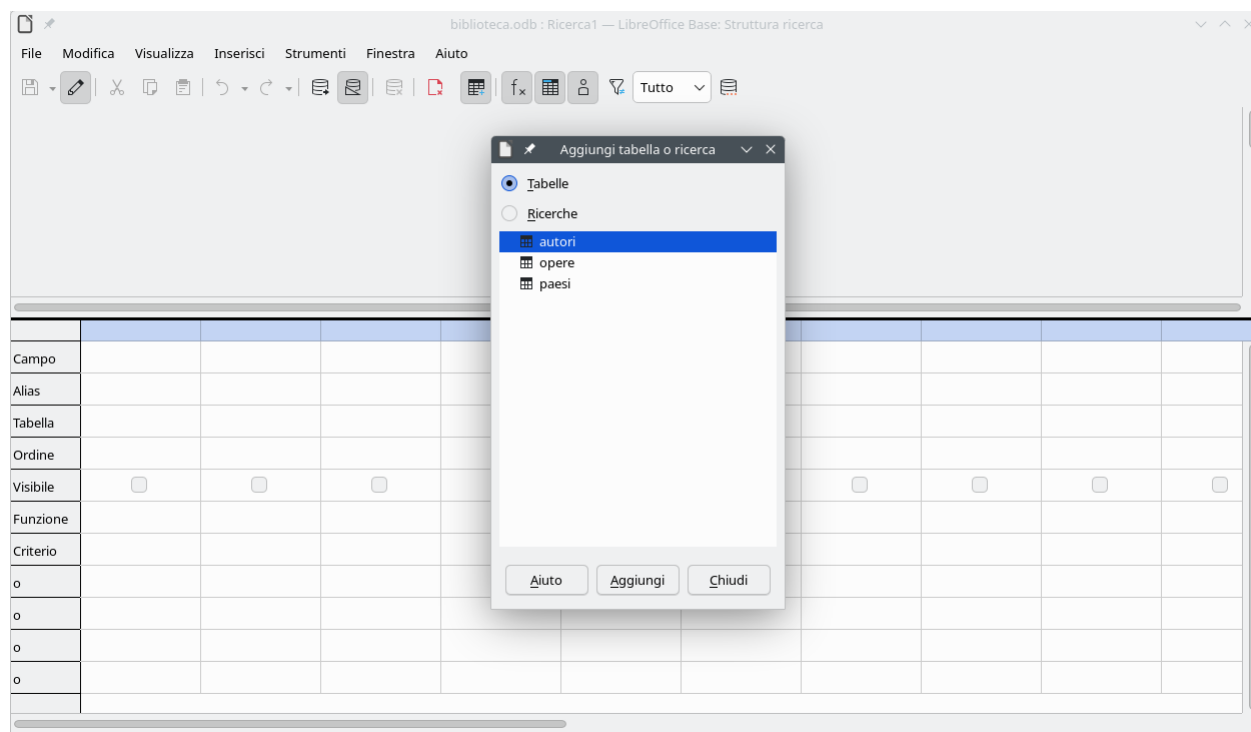
Avendo già predisposto le due ricerche chiamate `_autori` e `_paesi`, funzionali alla costruzione dei formulari nel precedente Capitolo, le troviamo elencate nella zona RICERCHE di questa finestra. Dal momento che queste due ricerche non ci servono ad altro, le abbiamo nominate con un underscore iniziale, che non metteremo sulle ricerche vere.

Nella zona delle attività ci vengono proposte tre modalità per fare una ricerca sul database:

- . CREA RICERCA IN VISTA STRUTTURA...
- . USA LA PROCEDURA GUIDATA PER LA CREAZIONE DI RICERCHE...
- . CREA RICERCA IN VISTA SQL...

3.1 Creazione di ricerca in vista struttura

Cliccando sulla prima modalità, CREA RICERCA IN VISTA STRUTTURA... ci troviamo di fronte questa finestra



Proponiamoci di creare una ricerca che ci fornisca nomi dell'autore e titolo di tutte le opere di autori italiani che abbiamo in biblioteca.

Nella finestrella che vediamo in primo piano abbiamo l'elenco delle tabelle che compongono il nostro database. Per la nostra ricerca ci interessano le tabelle dei paesi, degli autori e delle opere in quanto i dati che vogliamo estrarre le coinvolgono tutte e tre.

Selezioniamo la tabella `paesi` e clicchiamo su AGGIUNGI e facciamo così per le tabelle `autori` e `opere`. Fatto questo clicchiamo su CHIUDI.

Nella zona superiore della finestra ci ritroviamo gli schemi delle tre tabelle



Questi schemi ci servono per indicare le relazioni esistenti tra le tabelle che siano utili alla ricerca che vogliamo effettuare.

Nel nostro caso, dovendo estrarre il nome degli autori italiani, la prima relazione utile è quella tra idPaese della tabella dei paesi e l'idPaese della tabella degli autori. Con il mouse andiamo pertanto a collegare, per trascinamento, queste due voci negli schemi delle prime due tabelle (puntiamo con il mouse su idPaese della tabella dei paesi e, con premuto il tasto sinistro del mouse, lo trasciniamo su idPaese della tabella degli autori).

L'altra relazione utile, dovendo estrarre i titoli delle opere di questi autori, è quella tra idAutore della tabella degli autori e idAutore della tabella delle opere. Con il mouse andiamo pertanto a collegare queste due voci tra la seconda e la terza tabella.

Con questi interventi gli schemi delle tre tabelle appaiono ora con le relazioni evidenziate, così



Ora dobbiamo stabilire che cosa vogliamo e farne richiesta nella parte inferiore della finestra principale.

Il risultato della ricerca dovrebbe comparire come una tabella con due colonne, la prima contenente il nome dell'autore, la seconda contenente il titolo dell'opera, il tutto riferito ad autori italiani.

Andiamo allora nella prima colonna della finestra, con un click sulla riga CAMPO apriamo il menu a caduta e selezioniamo AUTORI.NOME.

Nella riga ALIAS possiamo indicare l'intestazione della colonna nella tabella che mostrerà il risultato della ricerca: se non lo facciamo, questa intestazione corrisponderà al nome della colonna nella tabella del database. Nel nostro caso indichiamo Autore, in quanto più esplicativo di Nome. Cliccando sul campo libero nella riga TABELLA esce la conferma della tabella degli autori.

Nella riga ORDINE possiamo scegliere che gli autori compaiano in ordine alfabetico crescente, quindi clicchiamo su crescente nel menu a tendina.

Nella riga VISIBILE, visto che il nome dell'autore vogliamo che compaia nella tabella, selezioniamo la casellina.

Andiamo ora nella seconda colonna e selezioniamo come CAMPO TITOLI.OPERE. Lasciamo perdere l'alias in quanto l'intestazione titolo è adatta. Lasciamo perdere l'ordinamento e scegliamo anche in questo caso VISIBILE.

Nella terza colonna inseriamo la condizione della ricerca che consiste nel fatto che il campo paese della tabella paesi deve contenere la stringa 'Italia'. Selezioniamo, nel solito modo, PAESI.PAESE nella riga del CAMPO. Facciamo in modo che il quadratino VISIBILE non sia selezionato, in quanto non ci interessa vedere il paese nel risultato della ricerca. Nella riga CRITERIO inseriamo la dizione COME 'Italia'.

Con click sul pulsante  eseguiamo la ricerca, il cui risultato compare nella zona superiore della finestra in questo modo

	Autore	titolo
▶	Manzoni, Alessandro	I promessi sposi
	Veronesi, Sandro	Caos calmo

Se riteniamo che ci possa servire ancora, memorizziamo questa ricerca dandole un nome che ne ricordi lo scopo, per esempio opere_per_paese_autore.

* * *

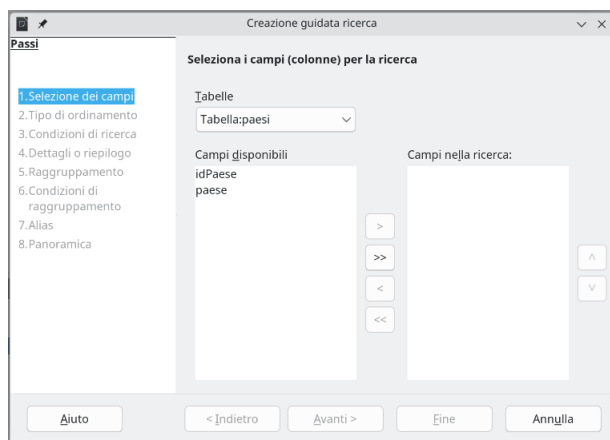
Le ricerche che abbiamo memorizzato, da qui in poi compariranno nel quadrante RICERCHE della finestra delle ricerche ed il loro risultato comparirà semplicemente facendo doppio click su di esse. Così facendo, tuttavia, il risultato rispecchierà l'impostazione che aveva la ricerca al momento della memorizzazione: l'ultima prodotta, per esempio, relativa all'elencazione di autori e opere per autori italiani, produrrà sempre questo risultato.

Essa, tuttavia, ci può essere utile per estrarre autori e opere per autori statunitensi: per arrivare a questo risultato basta che clicchiamo destro sul nome della ricerca e scegliamo MODIFICA nel menu a discesa che si apre. Nella successiva finestra sostituiamo USA a Italia nella condizione COME, clicchiamo sul pulsante di esecuzione e otteniamo l'elenco degli autori e delle opere per autori statunitensi. Al momento dell'uscita dalla finestra ci viene richiesto se memorizzare: possiamo farne a meno e tenerci il modello relativo all'Italia.

3.2 Creazione di ricerca con la procedura guidata

La procedura guidata può servire per ricerche molto semplici, tipo quella impostata nel Paragrafo 2.2 per ottenere l'elenco dei Paesi con rispettivo codice identificativo.

Cliccando sulla seconda modalità, USA LA PROCEDURA GUIDATA PER LA CREAZIONE DI RICERCHE..., apriamo questa finestra di dialogo



Agendo nella finestrella TABELLE ho già scelto la tabella paesi in quanto è quella implicata nella ricerca.

Dal momento che la tabella destinata a mostrare il risultato della ricerca deve contenere un elenco che accosta ai Paesi il relativo codice identificativo trasferiamo nei CAMPI DELLA RICERCA, nell'ordine, il campo paese e il campo idPaese e clicchiamo su AVANTI.

Nella successiva finestra di dialogo scegliamo nella prima finestrella per l'indicazione dell'ordinamento il dato paesi . paese confermando la scelta di default per un ordinamento crescente del risultato e clicchiamo su AVANTI.

Nella finestra successiva non abbiamo da indicare alcuna condizione di ricerca in quanto vogliamo l'elenco completo, pertanto la ignoriamo e clicchiamo su AVANTI.

Ripetiamo AVANTI per le due successive finestre, accettandone il contenuto come proposto, e nell'ultima finestra, inserito un nome per la ricerca nella finestrella relativa, clicchiamo su FINE.

3.3 Creazione di ricerca con il linguaggio SQL

SQL (Structured Query Language) è un linguaggio di programmazione standardizzato utilizzato per archiviare, gestire, modificare e recuperare informazioni all'interno dei database relazionali.

In questa sede ci basta conoscere il comando `select` di questo linguaggio, che serve per leggere i dati dalle tabelle del database e la cui sintassi è

```
select <nome_colonna>, <nome_colonna>, ... from <tabella>, <tabella>, ... where <condizioni>;
```

Questo è il comando più importante del linguaggio SQL e, per meglio fissare i concetti sulla sua sintassi sottolineo che

- dopo `select` va inserito, attraverso il nome di colonna, l'elenco dei campi che si vogliono evidenziare nel risultato della ricerca (in gergo la query), separati da virgola e nell'ordine in cui si vogliono evidenziare; non appena vi è il minimo rischio di ambiguità e, comunque, nel caso i campi da evidenziare non corrispondano, nell'ordine o nel numero, a quelli che sono coinvolti nella ricerca, il campo va indicato non semplicemente come `nome_colonna` ma nella forma più completa `tabella.nome_colonna`;
- dopo `from` va inserito l'elenco delle tabelle coinvolte nella ricerca, coinvolte o per l'estrazione di campi da evidenziare o semplicemente perché contengono almeno un elemento utile alla ricerca;
- dopo `where`, separate eventualmente da `and`, si indicano le relazioni tra le tabelle coinvolte nella ricerca nella forma `tabella.nome_colonna = tabella.nome_colonna` e poi, sempre con `and`, si aggiungono le condizioni di ricerca
 - nella forma `tabella.nome_colonna like '.....'` se abbiamo a che fare con dati espressi in forma di testo. Per esprimere la condizione `like` si possono usare i caratteri jolly che, nel linguaggio SQL sono:
 - _ per un solo carattere,
 - % per un numero imprecisato di caratteri;
 - nella forma `tabella.nome_colonna =` oppure `>` oppure `<` a un numero se abbiamo a che fare con dati numerici;
 - nella forma `tabella.nome_colonna is <valore>` se vogliamo esprimere un criterio di perfetta identità; `<valore>` può essere una stringa di testo (espressa tra apici), un numero o il valore `null`, scritto tale e quale senza apici oppure, sempre scritto senza apici, uno dei valori booleani `true` o `false`.

Sembra tutto molto complicato ma, dopo un po' di pratica, ci accorgiamo che non è così.

Per effettuare una ricerca utilizzando questo linguaggio clicchiamo sulla modalità CREA RICERCA IN VISTA SQL... e nell'editor che si apre davanti a noi scriviamo la query.

Per esempio, se vogliamo conoscere autore e titolo di opere scritte da autori viventi che abbiamo nella nostra biblioteca scriviamo

```
select nome, titolo from autori, opere where autori.idAutore = opere.idAutore and autori.morte is null
```

e clicchiamo sul pulsante  generiamo questo risultato

	nome	titolo
▶	Follett, Ken	Un luogo chiamato libertà
	Follett, Ken	I pilastri della terra
	Follett, Ken	Il codice Rebecca
	Veronesi, Sandro	Caos calmo

Se da menu FILE ► SALVA CON NOME salviamo questa ricerca dandole un nome, essa comparirà nell'elenco delle ricerche e potrà essere modificata come fosse stata creata in modalità vista struttura.

4 Rapporti

Dalla colonna di sinistra della finestra principale di Base possiamo scegliere una finestra di attività dedicata a RAPPORTI che ci propone la possibilità di creare rapporti sul contenuto del database in vista struttura o con procedura guidata.

Per la creazione in vista struttura è necessario siano installati sul computer il report builder, che è un'estensione di LibreOffice, e Java.

Entrambe le procedure sono macchinose e portano a risultati deludenti, soprattutto sul piano della libertà di esposizione, per cui le sconsiglio.

Molto più produttivo ritengo sia l'uso degli altri moduli di LibreOffice per inserire dati ricavati da Base in rapporti (Writer), tabelle (Calc), presentazioni (Impress) o disegni (Draw).

Nel Capitolo 1, creando il database, abbiamo accettato l'opzione proposta per default di registrare il database stesso.

Se non lo avessimo fatto potremmo farlo ora. Basterebbe aprire un qualsiasi modulo di LibreOffice ed agire con combinazione di tasti CTRL + SHIFT + F4 (in Writer e Calc è disponibile anche la voce di menu VISUALIZZA > SORGENTE DATI). In questo modo, appena sopra l'area di lavoro, si apre una finestrella contenente l'elenco dei database registrati. Se quello che ci interessa non c'è e lo vogliamo inserire, clicchiamo destro nella zona dove c'è l'elenco dei database registrati e scegliamo DATABASE REGISTRATI dal menu a tendina. Nella successiva finestra di dialogo clicchiamo su NUOVO e inseriamo il database che vogliamo registrare seguendo i vari passaggi guidati.

Il database inserito compare con la possibilità di visualizzare gli elenchi delle ricerche e delle tabelle che contiene. Per trascinamento con il mouse, a partire da questi elenchi, possiamo inserire i valori di ricerche e tabelle nell'area di lavoro del modulo LibreOffice in cui vogliamo riportare questi valori, con tutte le possibilità di titolazione e commento collaterale, formattazione, colorazione e quant'altro ci consenta il modulo stesso (se siamo in Calc anche elaborazioni). Ovviamente dobbiamo prima costruire e memorizzare le tabelle o le ricerche che ci servono.

Parte II

MariaDB

MariaDB è un fork del software MySQL creato dallo stesso autore principale di quest'ultimo, il finlandese Ulf Michael Widenius, detto "Monty", che ha inserito in entrambi il nome di sue figlie: My e Maria.

La prima versione di MySQL nasce ad opera del Widenius e di David Axmark nel 1996 e la distribuzione avviene ad opera della società MySQL AB, di cui Widenius diviene chief technical officer.

Nel 2008 la società viene acquisita dalla Sun Microsystems e a seguito di questa acquisizione Widenius vorrebbe che lo sviluppo e il perfezionamento del software si aprisse alla comunità.

Ma non solo questo desiderio è disatteso, il che causa le dimissioni del Widenius dalla Sun, ma la Sun, nel 2009 viene acquisita dalla Oracle che solo parzialmente apre all'open source, prevedendo una doppia licenza (GPL + commerciale Oracle), dopo il lancio, da parte del Widenius, della campagna Help saving MySQL, in cui si chiedeva alle autorità antitrust europee di non autorizzare l'acquisizione.

La sola parziale vittoria della doppia licenza è la causa della nascita di MariaDB, che usa solo la licenza GPL, senza ambiguità legali ed è software libero in piena regola.

Ovviamente MariaDB sostituisce gradualmente MySQL nei repository delle distribuzioni Linux, guarda caso nato ad opera del connazionale di Widenius, il poco più giovane Linus Torvalds.

Dopo questo omaggio ai finlandesi del software libero vediamo di cosa si tratta.

MariaDB è un motore di database client/server e, nella compagine dei suoi simili (MySQL, DB2, Oracle, PostgreSQL, SQL Server), alla pari del fratello MySQL, si pone come più semplice da gestire e alla mano, poco costoso o di nessun costo, sia pure meno adatto a gestire moli di dati molto elevate.

In questo manuale mi propongo di presentare alcuni modi di utilizzare MariaDB, dando per scontato che il lettore sappia, spero aiutato da quanto ho detto nell'Introduzione, in che cosa consista un database, in particolare un database relazionale.

MariaDB crea e gestisce database relazionali.

Lo troviamo all'indirizzo <https://mariadb.org/>.

Nella finestra DOCUMENTATION di questo sito troviamo il manuale completo in formato PDF, che è di ben 4200 pagine. Il che la dice lunga su ciò che ci troviamo di fronte.

Ma in questo manuale di solo qualche decina di pagine penso di mettere in grado chiunque di installare MariaDB e di cominciare ad usarlo creando e gestendo qualche piccolo database. Chi si appassioni può via via consultare il citato manuale.

1 Caratteristiche

Come detto in premessa, MariaDB eredita tutte le caratteristiche di MySQL e il suo essere più semplice e meno costoso dei suoi simili più complicati e adatti alla gestione di grandi database lo rende ideale per compiti circoscritti, come quello di supportare un server web su cui vengano gestite elaborazioni server side utilizzando linguaggi come PHP, con la necessità di memorizzare i dati per le elaborazioni stesse.

Ciò fa sì che la documentazione ricorrente su MariaDB, come quella su MySQL, presenti quasi sempre questi software in blocco con i software di un server web come Apache2, su cui possa avvenire l'elaborazione con linguaggi come PHP. Al punto da far pensare che MariaDB e MySQL, per funzionare, abbiano bisogno di Apache2 e del linguaggio PHP.

Nulla di più sbagliato, perché MariaDB, come MySQL, di cui da adesso non parlerò più in quanto MariaDB e MySQL sono praticamente la stessa cosa, vivono di vita propria, hanno un server proprio e riconoscono innanzi tutto i comandi del linguaggio SQL (Structured Query Language), che è lo standard per operare con i database.

Questo manualetto è destinato a fare chiarezza anche su questo.

2 MariaDB a riga di comando

Cominciamo a vedere MariaDB in quanto tale: un DBMS (Data Base Management System) client/server a riga di comando, cioè operante con comandi a terminale (quello che in Windows si chiama prompt dei comandi).

2.1 Installazione

Il sito dove troviamo MariaDB è all'indirizzo <https://mariadb.org/>

Aperto la finestra DOWNLOAD abbiamo modo di scegliere se scaricare il source code o installer per Linux e Windows.

Ma se lavoriamo su Linux o Mac abbiamo strade più semplici, in quanto il software MariaDB è presente nei relativi repository.

Sistema Linux

Il comando a terminale per l'installazione è

```
sudo apt install mariadb-server
```

se siamo su Debian e derivate (Ubuntu, MXLinux, Mauna Linux, Linux Mint, ecc.),

```
sudo dnf install mariadb-server
```

se siamo su Red Hat e derivate (Fedora, OpenSUSE, Open Mandriva, ecc.).

Subito dopo l'installazione dobbiamo avviare il servizio MariaDB con il comando a terminale

```
sudo systemctl start mariadb
```

e abilitare MariaDB all'avvio del sistema con il comando a terminale

```
sudo systemctl enable mariadb
```

Infine è necessaria la configurazione di sicurezza con il comando a terminale

```
sudo mysql_secure_installation
```

che ci sottopone ad una serie di quesiti:

- . al primo di questi, Enter current password for root:

se siamo alla prima installazione e non abbiamo ancora una password per l'utente

root di MariaDB non scriviamo nulla e premiamo INVIO,

- . a tutti gli altri rispondiamo y,

- . quando ci viene richiesto indichiamo la password per l'utente root.

Finalmente possiamo accedere alla shell di MariaDB con il comando a terminale

```
mariadb -u root -p
```

digitando poi la password che abbiamo impostato prima.

Sistema Mac

Il procedimento è molto simile a quello per il sistema Linux.

In questo caso il gestore di pacchetti che utilizziamo è Homebrew e il comando a terminale per l'installazione è

```
brew install mariadb
```

L'avvio del servizio avviene con il comando a terminale


```
brew services start mariadb
```

cui far seguire il comando a terminale

```
mysql_secure_installation
```

di fronte ai cui quesiti ci comportiamo come detto per Linux.

L'accesso alla shell di MariaDB avviene anche qui con il comando

```
mariadb -u root -p
```

Sistema Windows

Dalla pagina **DOWNLOAD** del sito di MariaDB scarichiamo il file `.msi` e lo avviamo per l'installazione.

Accettato l'accordo di licenza selezioniamo le componenti da installare (solitamente sono sufficienti quelle predefinite), impostiamo, quando richiesto, la password per l'utente root (passaggio importantissimo), configuriamo MariaDB come servizio Windows («Install as service») e completiamo l'installazione cliccando su **INSTALL** e poi su **FINISH**.

Con il comando

```
mariadb -u root -p
```

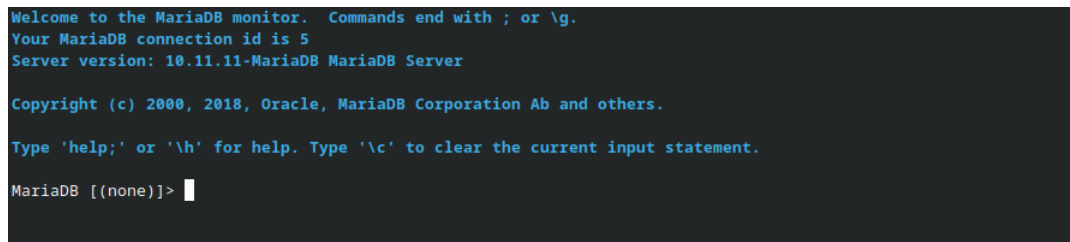
nel prompt dei comandi apriamo la shell di MariaDB.

2.2 Uso di MariaDB a riga di comando

Con il comando a terminale (prompt dei comandi in Windows)

```
mariadb -u root -p
```

e inserita la password per l'utente root apriamo la shell di MariaDB che, sul mio sistema Fedora su cui sto scrivendo, si presenta così



```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 10.11.11-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

In questa shell possiamo chiedere a MariaDB di fare ciò che ci serve e lo facciamo utilizzando l'unico linguaggio che MariaDB capisce: il linguaggio SQL (Structured Query Language).

Si tratta di un linguaggio molto ricco e ben documentato da numerosi manuali su carta e online. Il manuale di MariaDB è voluminoso anche perché contiene una completa documentazione su questo linguaggio.

Qui richiamo i comandi più ricorrenti.

L'inserimento dei comandi può indifferentemente avvenire con lettere maiuscole o minuscole. I comandi si possono scrivere su più righe, andando a capo con INVIO, e terminano con il punto e virgola ;.

Per creare un nuovo database usiamo il comando

```
CREATE DATABASE <nome>;
```

Per mostrare tutti i database presenti usiamo il comando

```
SHOW DATABASES;
```

Per eliminare un database esistente usiamo il comando

```
DROP DATABASE <nome>;
```

Per scegliere il database su cui lavorare usiamo il comando

```
USE <nome_database>;
```

Una volta scelto il database su cui lavorare abbiamo a disposizione i seguenti comandi.

Per creare una tabella

```
CREATE TABLE <nome_tabella> (<nome_colonna><tipo_dato>[attributi], <nome_colonna><tipo_dato>[attributi], ...);
```

I tipi di dato definibili in MariaDB sono moltissimi ma per i nostri usi dilettanteschi bastano questi

INT per numero intero

REAL per numero decimale

TEXT per testo

Gli attributi sono di inserimento facoltativo e i più frequentemente usati sono:

PRIMARY KEY, che rende il campo la chiave primaria della tabella (quella che serve per instaurare relazioni con altre tabelle);

NOT NULL, che rende il campo obbligatoriamente destinato a contenere un dato, contrariamente a quella che sarebbe l'impostazione di default;

AUTO_INCREMENT, destinato solo a campi contenenti un numero per fare in modo che il contenuto del campo si incrementi automaticamente di una unità a partire dal più grande valore già presente. Attributo molto utile per la primary key.

Per inserire dati in una tabella

```
INSERT INTO <nome_tabella> (<colonna_interessata>, <colonna_interessata>, ...) VALUES (<valore>, <valore>, ...);
```

Ovviamente i valori vanno elencati nello stesso ordine delle colonne. I valori numerici si inseriscono tali e quali, mentre i valori espressi come testo vanno racchiusi tra semplici apici '.

Per cancellare una tabella

```
DROP TABLE <nome_tabella>;
```

Per eliminare righe da una tabella

```
DELETE FROM <nome_tabella> WHERE <condizioni>;
```

Per modificare dati in una tabella

```
UPDATE <nome_tabella> SET <nome_colonna> = <valore> WHERE <condizioni>;
```

Per leggere i dati dalle tabelle del database

```
SELECT <nome_colonna>, <nome_colonna>, ... FROM <tabella>, <tabella>, ... WHERE <condizioni>;
```

Questo è il comando più importante del linguaggio SQL e, per meglio fissare i concetti sulla sua sintassi sottolineo che

- dopo SELECT va inserito, attraverso il nome di colonna, l'elenco dei campi che si vogliono evidenziare nel risultato della ricerca (in gergo la query), separati da virgola e nell'ordine in cui si vogliono evidenziare; non appena vi è il minimo rischio di ambiguità e, comunque, nel caso i campi da evidenziare non corrispondano, nell'ordine o nel numero, a quelli che sono coinvolti nella ricerca, il campo va indicato non semplicemente come nome_colonna ma nella forma più completa tabella.nome_colonna;
- dopo FROM va inserito l'elenco delle tabelle coinvolte nella ricerca, coinvolte o per l'estrazione di campi da evidenziare o semplicemente perché contengono almeno un elemento utile alla ricerca;
- dopo WHERE, separate eventualmente da AND, si indicano le relazioni tra le tabelle coinvolte nella ricerca nella forma tabella.nome_colonna = tabella.nome_colonna e poi, sempre con AND, si aggiungono le condizioni di ricerca
 - nella forma tabella.nome_colonna like '.....' se abbiamo a che fare con dati espressi in forma di testo. Per esprimere la condizione like si possono usare i caratteri jolly che, nel linguaggio SQL sono:
 - _ per un solo carattere,
 - % per un numero imprecisato di caratteri;

- nella forma `tabella.nome_colonna =` oppure `>` oppure `<` a un numero se abbiamo a che fare con dati numerici;
- nella forma `tabella.nome_colonna is <valore>` se vogliamo esprimere un criterio di perfetta identità; `<valore>` può essere una stringa di testo (espressa tra apici), un numero o il valore null, scritto tale e quale senza apici oppure, sempre scritto senza apici, uno dei valori booleani true o false.

Sembra tutto molto complicato ma, dopo un po' di pratica, ci accorgiamo che non è così.

* * *

MariaDB è un database che può essere usato, anche contemporaneamente, da più utenti: è questo il grande vantaggio dei database client/server.

Quando l'abbiamo installato, come abbiamo visto nel precedente paragrafo, abbiamo creato un utente, chiamato `root`, che ha il potere (nel linguaggio dei database «privilegio») di fare tutto ciò che si può fare con il database, compreso quello di creare nuovi utenti assegnando anche a loro il potere di fare tutto o di fare solo alcune cose.

Per creare un nuovo utente, nella shell di MariaDB aperta dall'utente `root`, si usa il comando `create user '<nome_utente>' @ 'localhost' identified by <password>;`

Per assegnare gli stessi privilegi dell'utente `root` ad un altro utente, sempre nella shell di MariaDB aperta dall'utente `root`, si usa il comando

`grant all privileges on *.* to '<nome_utente>' @ 'localhost';`

Per assegnare privilegi limitati, al posto di `all privileges` si scrivono le azioni consentite utilizzando il nome del relativo comando SQL (`insert`, `update`, `select`, ecc.), separate da virgola se sono più di una e al posto del primo asterisco dopo `on` si scrive il nome del database per cui sono consentite queste azioni.

Infine si conferma con `flush privileges;`

Un esercizio istruttivo

Supponiamo di avere una biblioteca di quattro libri e, anche se totalmente inutile per padroneggiarla ma molto utile come esercizio in questa sede, vogliamo costruire un database per gestirla.

I libri sono

I promessi sposi di Alessandro Manzoni

Un luogo chiamato libertà di Ken Follett

I pilastri della terra di Ken Follett

Furore di John Steinbeck.

Il primo passo, sicuramente il più importante, è la progettazione del database e, per affrontarlo, dobbiamo innanzi tutto chiederci a cosa vogliamo che serva questo database. Nel nostro caso supponiamo che le esigenze siano: sapere le opere che abbiamo dei vari autori, sapere le opere che abbiamo scritte in certi periodi, sapere gli autori che abbiamo rappresentati nella nostra biblioteca per Paesi di appartenenza, sapere quali opere abbiamo di autori di un certo Paese.

Da qui deriviamo che i dati che dovremo inserire nel database saranno: i nomi degli autori, i titoli delle opere, le date di nascita e di morte degli autori, i Paesi di appartenenza degli autori.

Dal momento che la regola d'oro del buon database è «scrivere le cose una ed una sola volta», sia pure per colpa del solo Ken Follett, di cui abbiamo due libri, non possiamo ricorrere ad una tabella gestita in un foglio di calcolo in quanto dovremmo scrivere due volte il nome di Ken Follett, due volte la sua data di nascita, due volte il nome del Paese di appartenenza. Dobbiamo allora ricorrere ad un database relazionale e MariaDB è qui per questo.

Una organizzazione ottimale del database che assolvere alle finalità che ci siamo posti potrebbe essere quella di impostare il database su tre tabelle: una per gli autori, contenente le date di nascita e di morte, una per i Paesi di appartenenza e una per i titoli delle opere. La tabella degli autori si dovrà relazionare con quella dei Paesi e con quella dei titoli delle opere e quest'ultima basterà che si relazioni con quella degli autori, attraverso la quale finirà per essere relazionata anche con la tabella dei Paesi.

La tabella autori dovrà pertanto avere cinque colonne: idAutore, che assolverà la funzione di chiave primaria e conterrà il numero di riferimento corrispondente all'autore, nome, che conterrà il nome dell'autore, nascita, che conterrà la data di nascita dell'autore, morte, che conterrà la data di morte dell'autore e idPaese, che conterrà il numero di riferimento del Paese derivante dalla tabella dei Paesi.

La tabella paesi avrà due colonne: idPaese, che assolverà la funzione di chiave primaria e conterrà il numero di riferimento corrispondente al Paese e paese, contenente il nome del Paese.

La tabella titoli avrà tre colonne: idTitolo, che assolverà la funzione di chiave primaria e conterrà il numero di riferimento corrispondente al titolo di ciascuna opera, titolo, che conterrà il titolo dell'opera e idAutore, che conterrà il numero di riferimento corrispondente all'autore derivante dalla tabella autori.

La progettazione è finita ed ora dobbiamo passare alla realizzazione.

Apriamo la shell di MariaDB e creiamo il database, chiamandolo «biblioteca», con il comando `create database biblioteca;` e selezioniamolo per essere usato con `use biblioteca;`

Il prompt della shell di MariaDB diventa `MariaDB [biblioteca]>`

e ciò che faremo da qui in poi sarà riferito al database «biblioteca».

Ora creiamo la prima delle tre tabelle che ci servono, che chiamiamo «autori», con il comando `create table autori (idAutore int primary key auto_increment, nome text, nascita int, morte int, idPaese int;`

Poi creiamo la tabella dei Paesi, che chiamiamo «paesi», con il comando: `create table paesi (idPaese int primary key auto_increment, paese text);`

Infine creiamo la tabella delle opere, che chiamiamo «titoli», con il comando `create table titoli (idTitolo int primary key auto_increment, titolo text, idAutore int);` sempre non dimenticando di chiudere i comandi con il punto e virgola.

Se facciamo qualche errore o scriviamo parole che MariaDB non comprende sarà sua cura avvisarci; se vogliamo comunque verificare che tutto sia andato bene possiamo scrivere il comando `show tables;`

e ciò che ci viene mostrato ci conforta

```
MariaDB [biblioteca]> show tables;
+-----+
| Tables_in_biblioteca |
+-----+
| autori                |
| paesi                 |
| titoli                |
+-----+
```

Non ci resta che inserire i dati. Questa è la cosa più brutta da fare lavorando da terminale e nei capitoli successivi vedremo sistemi più comodi per farlo. Ma questo esercizio serve per capire come lavora MariaDB ed ha la sua utilità.

Cominciamo ad inserire i Paesi, visto che la chiave del Paese (idPaese) ci servirà per alimentare la tabella degli autori, e lo facciamo con i comandi

```
insert into paesi (paese) values ('Italia');
insert into paesi (paese) values ('Gran Bretagna');
```

```
insert into paesi (paese) values ('USA');
```

Abbiamo inserito i paesi, rispettivamente, di Alessandro Manzoni, di Ken Follett e di John Steinbeck, senza inserire il dato della prima colonna, idPaese, che avevamo definito autoincrementale.

Se scriviamo il comando

```
select idPaese, paese from paesi;
```

chiedendo a MariaDB di selezionare i dati contenuti nelle colonne idPaese e paese della tabella paesi, otteniamo la risposta

```
MariaDB [biblioteca]> select idPaese, paese from paesi;
+-----+-----+
| idPaese | paese |
+-----+-----+
| 1 | Italia |
| 2 | Gran Bretagna |
| 3 | USA |
+-----+-----+
```

a dimostrazione del fatto che MariaDB ha attribuito automaticamente i valori 1, 2 e 3, nell'ordine, a Italia, Gran Bretagna e USA.

Inseriamo ora gli autori, con i comandi

```
insert into autori (nome,nascita,morte,idPaese) values ('Manzoni, Alessandro', 1785, 1873, 1);
```

```
insert into autori (nome, nascita, morte, idPaese) values ('Follett, Ken', 1949, null, 2);
```

```
insert into autori (nome, nascita, morte, idPaese) values ('Steinbeck, John', 1902, 1968, 3);
```

sempre trascurando l'inserimento del dato nella prima colonna, quella dell'idAutore, in quanto la fa MariaDB, dando automaticamente l'identificativo 1 a Manzoni, 2 a Follett e 3 a Steinbeck, nell'ordine con cui li abbiamo inseriti. Dal momento che Ken Follett è vivente, in luogo della data di morte abbiamo inserito il valore null. Nella colonna idPaese abbiamo inserito gli identificativi di Paese desunti dalla relativa tabella. Notare che, mentre i valori numerici si inseriscono tali e quali, i testi (stringhe) vanno racchiusi tra apici.

Finalmente tocca alle opere, che inseriamo con i comandi

```
insert into titoli (titolo, idAutore) values ('I promessi sposi', 1);
```

```
insert into titoli (titolo, idAutore) values ('Un luogo chiamato libertà', 2);
```

```
insert into titoli (titolo, idAutore) values ('I pilastri della terra', 2);
```

```
insert into titoli (titolo, idAutore) values ('Furore', 3);
```

ed abbiamo così creato il database della nostra biblioteca.

A questo punto vediamo come possiamo utilizzarlo per avere alcune informazioni sulla nostra raccolta letteraria.

Per conoscere le opere scritte da Ken Follett, dobbiamo scrivere nella shell di MariaDB questo comando, che nel linguaggio dei database si chiama query:

```
select titolo from titoli, autori where titoli.idAutore = autori.idAutore and autori.nome like 'Follett%';
```

ed otteniamo questo risultato

```
MariaDB [biblioteca]> select titolo from titoli, autori where titoli.idAutore = autori.idAutore and autori.no
me like 'Follett%';
+-----+
| titolo |
+-----+
| Un luogo chiamato libertà |
| I pilastri della terra |
+-----+
```

Nella query, in linguaggio SQL, chiediamo di selezionare il titolo delle opere ricorrendo alle tabelle titoli e autori, relazionate tra loro dall'identificativo dell'autore presente in entrambe e in corrispondenza di un autore il cui nome comincia con Follett. Puntualmente MariaDB ci restituisce l'elenco delle opere di Ken Follett che abbiamo in biblioteca.

Per conoscere le opere scritte da autori americani, in questo spezzone di schermata di shell

```
MariaDB [biblioteca]> select nome, titolo from autori, titoli, paesi where titoli.idAutore = autori.idAutore
-> and autori.idPaese = paesi.idPaese and paesi.paese like 'USA';
+-----+-----+
| nome          | titolo |
+-----+-----+
| Steinbeck, John | Furore |
+-----+-----+
```

abbiamo query e risultato. Nella query, in linguaggio SQL, chiediamo di selezionare autore e titolo delle opere, ricorrendo alle tabelle autori, titoli e paesi, autori e titoli relazionate tra loro dall'identificativo dell'autore presente in entrambe e autori e paesi relazionate tra loro dall'identificativo del Paese presente in entrambe, in corrispondenza di un paese che si chiama USA. E MariaDB ci indica l'unica opera scritta da un autore americano che abbiamo in biblioteca, come richiesto, con il nome dell'autore e il titolo.

Nel caso che segue abbiamo chiesto autore e titolo di opere scritte da autori nati prima del 1900

```
MariaDB [biblioteca]> select nome, titolo from autori, titoli where autori.idAutore = titoli.idAutore and aut
ori.nascita < 1900;
+-----+-----+
| nome          | titolo |
+-----+-----+
| Manzoni, Alessandro | I promessi sposi |
+-----+-----+
```

ed abbiamo puntualmente ottenuto la risposta.

Ora abbiamo voluto conoscere autore e titolo di opere scritte da autori viventi

```
MariaDB [biblioteca]> select nome, titolo from autori, titoli where autori.idAutore = titoli.idAutore and aut
ori.morte is null;
+-----+-----+
| nome          | titolo |
+-----+-----+
| Follett, Ken | Un luogo chiamato libertà |
| Follett, Ken | I pilastri della terra |
+-----+-----+
```

ed otteniamo l'elenco delle opere di Follett, l'unico vivente che abbiamo in biblioteca.

* * *

Finora abbiamo ottenuto risposte sullo schermo.

Se volessimo ottenere risposte su supporto meno volatile, o per conservarle o per stamparle, possiamo farlo da terminale con il comando

```
mariadb -u root -p <nome_database> -e 'query' > 'path_al_file.csv'
```

Sicché per scrivere su un file «opere_autori_viventi» in formato csv l'ultima query che abbiamo esemplificato scrivo a terminale

```
mariadb -u root -p biblioteca -e 'select nome, titolo from autori, titoli
where autori.idAutore = titoli.idAutore and autori.morte is null'
> '/usr/vittorio/Documenti/opere_autori_viventi.csv'
```

e nella cartella Documenti della mia home su Linux trovo il risultato della query su file .csv.

* * *

Altra cosa utile può essere esportare il database su un file in formato SQL, o come backup o per utilizzarlo su altro computer.

Per farlo, sul normale terminale diamo il comando

```
mariadb-dump -u root -p <nome_database> > 'path_al_file.sql'
```

Sicché per esportare il database che abbiamo costruito scrivo a terminale

```
mariadb-dump -u root -p biblioteca > '/home/vittorio/Documenti/biblioteca.sql'
```

e nella cartella Documenti della mia home su Linux trovo il file .sql del database.

Per importare il database su un altro computer, dopo aver copiato su quel computer il file .sql, occorre creare nella shell di MariaDB su quel computer il database di destinazione dell'importazione e poi, a terminale normale, scrivere il comando

```
mariadb -u root -p <database_di_destinazione> < 'path_al_file.sql'
```

3 Uso di MariaDB con LibreOffice Base

Un modo egregio che ci offre il mondo del software libero per lavorare comodamente con MariaDB è di farlo attraverso il modulo Base di LibreOffice (o delle un po' meno libere edizioni OpenOffice). In questo modo, una volta creato il database e impostate le sue tabelle con il software MariaDB a riga di comando che abbiamo visto nel precedente Capitolo lo colleghiamo al modulo Base di LibreOffice e, da qui in poi, alimentazione del database, ricerche e quant'altro lo facciamo con tutte le potenti funzionalità che ci offre Base: a partire dalla possibilità di alimentare il database con comode maschere di inserimento fino a quella di produrre risultati di ricerche ben impaginati e inseribili in relazioni dall'aspetto professionale.

Ci si può chiedere: ma, dal momento che il modulo Base può lui stesso creare un database, che bisogno c'è di dargli in pasto un database di altra natura come MariaDB?

La risposta è semplice: il database creato da Base è utilizzabile solo con Base mentre il database creato con MariaDB è utilizzabile ovunque e trasferibile creandone un salvataggio su file in formato SQL con il comando che abbiamo appena visto alla fine del Capitolo precedente.

L'ottenimento di tutti questi benefici costa ovviamente qualche fatica, che ritengo comunque sia ben compensata dai risultati: prima fra tutte la fatica di predisporre il computer ad instaurare una connessione tra database e altre applicazioni.

Open DataBase Connectivity (ODBC)

Il sistema ODBC ritengo sia relativamente il più semplice ed il più stabile per creare una connettività di database con altre applicazioni: l'alternativa, peraltro molto efficiente nel caso di MariaDB, sarebbe la Java DataBase Connectivity (JDBC) che mi sembra più complicata.

Per la creazione di un contesto di connettività i problemi da superare sono due:

- sul computer devono essere installati i driver per il collegamento ad un certo tipo di database, nel nostro caso MariaDB,
- il computer deve darci la possibilità di creare ed evidenziare l'origine dei dati (Data Source Name, DSN) per instaurare il collegamento.

Sistema operativo Debian e derivate (Linux Ubuntu, Linux Mint, MXLinux, Mauna Linux, ecc.)

Sul sistema deve essere installato il package `unixodbc`. Lo si può verificare attraverso la presenza, in `/etc` dei file `odbc.ini` e `odbcinst.ini`.

Se non è installato lo installiamo con il comando a terminale

```
sudo apt install unixodbc
```

Poi è necessario installare i driver per MariaDB e lo facciamo con i comandi a terminale

```
sudo apt install odbc-mariadb
```

```
sudo apt install libreoffice-sdbc-mysql
```

Sistema operativo Red Hat e derivate (Fedora, OpenSUSE, ecc.)

Per installare su questi sistemi il package `unixODBC` (qui si chiama così), se non lo è ancora, diamo il comando a terminale

```
sudo dnf install unixODBC
```

Per i driver basta dare il seguente comando a terminale

```
sudo dnf install mariadb-connector-odbc
```

Sistema operativo Mac OS

Fratello di Linux per la comune derivazione da Unix, anche Mac OS lavora con quello che qui si chiama iODBC, che dovrebbe essere già installato insieme ad una utilità di interfaccia grafica ODBCManager, attraverso la quale possiamo creare i DSN.

Come in Linux abbiamo i file odbcinst.ini e odbc.ini, che si trovano nella directory /Library/ODBC.

Per poter lavorare con LibreOffice dobbiamo installare il MySQL Connector/ODBC che troviamo a questo indirizzo

<https://dev.mysql.com/downloads/connector/odbc/>

in un file DMG.

Dopo l'installazione avremo i file driver in genere in /usr/local/lib/.

Apriamo ODBCManager e registriamo il driver.

A questo fine andiamo su DRIVERS, clicchiamo su ADD, compiliamo DESCRIPTION: MySQL ODBC 8 e DRIVER FILE: /usr/local/lib/libmyodbc8w.so e clicchiamo su SALVA.

Sempre in ODBCManager ora creiamo il DSN (Data Source Name), andando su User DSN (o System DSN), cliccando su ADD e selezionando il driver appena registrato.

Compilati i campi clicchiamo su SALVA.

Possiamo verificare il risultato di questa impresa constatando che i DSN siano salvati in ~/Library/ODBC/odbc.ini e che i driver siano salvati in /Library/ODBC/odbcinst.ini

Sistema operativo Windows

All'indirizzo

<https://dev.mysql.com/downloads/connector/odbc/>

troviamo il Connector/ODBC in un file MSI che installiamo.

Ora da PANNELLO DI CONTROLLO ► STRUMENTI DI AMMINISTRAZIONE ► ORIGINE DATI ODBC andiamo a creare un nuovo DSN.

A questo scopo andiamo alla scheda DSN UTENTE (per un singolo utente) o DSN DI SISTEMA (per tutti gli utenti/servizi) e clicchiamo su AGGIUNGI.

Selezioniamo il driver scegliendo dall'elenco quello appena installato e clicchiamo su FINE.

Seguiamo poi la procedura guidata per inserire i Dettagli di Connessione

Possiamo verificare la connessione cliccando su TEST ORIGINE DATI per verificare che i parametri siano corretti e che il server sia raggiungibile.

Se tutto va bene clicchiamo OK e poi FINE per salvare la configurazione. successiva

Collegiamo il database

Lanciamo LibreOffice Base e, nella finestra che si apre, intitolata CREAZIONE GUIDATA DATABASE, scegliamo di procedere con COLLEGA A UN DATABASE ESISTENTE. Nella finestrella sottostante scegliamo MYSQL/MARIADB dall'elenco che ci viene proposto e clicchiamo su SUCCESSIVO.

Nella nuova finestra scegliamo CONNETTI IN MODO DIRETTO (USANDO IL CONNETTORE MARIADB) e clicchiamo su SUCCESSIVO.

Nella nuova finestra configuriamo la connessione inserendo il nome del database da collegare (biblioteca) e il server (localhost).

Nel seguito ci verrà chiesto il nome utente, che indichiamo cliccando sull'opzione PASSWORD RICHiesta e clicchiamo su VERIFICA LA CONNESSIONE.

Inserita poi la password per l'accesso al database, cliccando su OK instauriamo la connessione.

Arriviamo così alla finestra finale dove ci viene proposto di registrare o meno il database in LibreOffice (consiglio di scegliere SÌ e dirò subito il perché). Possiamo anche scegliere se aprire subito

il database o no. Cliccando su FINE creiamo un database di LibreOffice Base collegato al database sul server MariaDB, che sarà un file con estensione .odb e che sceglieremo dove memorizzare.

La registrazione del database in LibreOffice consente di collegare il database non solo al modulo Base ma anche agli altri moduli di LibreOffice, con utilità che vedremo nel seguito. Se non lo facciamo subito abbiamo comunque modo di registrare il database anche in un momento successivo.

Se abbiamo operato bene, ora troviamo memorizzato, nella directory che abbiamo scelto prima, un file `biblioteca.odb`, che è l'impalcatura creata con LibreOffice attorno al database MariaDB.

Aperto il file .odb così creato possiamo lavorarlo secondo quanto abbiamo visto nella Parte I di questo manuale, come fossimo in presenza di un database creato con Base.

4 Collegamenti attraverso i principali linguaggi di programmazione.

Tutti i principali linguaggi di programmazione (C/C++, Java, PHP e Python) hanno la possibilità, previa installazione di apposite librerie, di instaurare collegamenti destinati ad interagire con il motore MariaDB per creare database, tabelle, alimentarle, leggerle.

Ciò avviene instaurando un collegamento con il database, che è la cosa più difficile, e creare uno strumento, generalmente detto cursore, per passare al database collegato istruzioni in linguaggio SQL.

Lasciamo C, C++ e Java ai professionisti in quanto di quelli sopra citati sono i più ostici, a partire, per quanto riguarda C e C++ dalla necessità di procurarsi l'header giusto, sapere dove metterlo, ecc.

Come sempre avviene, è il linguaggio Python quello con cui si può affrontare questa fatica nella maniera più semplice.

La libreria da installare si chiama `pymysql` (nel mondo Red Hat chiamato anche `mysql-connector-python`).

Utilizzo di Python3

Una volta importato il modulo con

```
import pymysql
```

dobbiamo configurare la connessione indicando dove si trova il database e con quale password vi si può accedere e lo facciamo con questo codice

```
config = {
    'user': '<nome_utente>',
    'password': '<password>',
    'host': '127.0.0.1',
    'database': '<nome_database>',
    'port': 3306
}
```

dove per `<nome_utente>`, `<password>` e `<nome_database>` indichiamo ciò che serve.

Poi creiamo la connessione e un cursore per poter veicolare le istruzioni in linguaggio SQL con questo codice

```
connessione = pymysql.connect(**config)
cursore = connessione.cursor()
```

Per lavorare sul database con cui siamo collegati, utilizziamo ora il metodo `execute` del cursore per passare le istruzioni in linguaggio SQL:

```
cursore.execute("<comando_SQL>")
```

esegue un comando SQL indicato come stringa o come variabile stringa.

A questo punto, se le istruzioni SQL consistevano in una query destinata a produrre un risultato, recuperiamo questo risultato con

```
risultato = cursore.fetchall()
```

creando una lista, chiamata risultato, nella quale ogni elemento è una tupla che rappresenta la riga di una astratta tabella con i dati della query per riga e colonna, come estratti dal database su cui è stata fatta la ricerca, dati a loro volta rappresentati sotto forma di stringhe unicode.

Alla fine del nostro lavoro dobbiamo chiudere la connessione con

```
connessione.close().
```

Facciamo un piccolo esempio: ci proponiamo di estrarre il contenuto della tabella dei Paesi del database «biblioteca» che abbiamo creato e alimentato per esercizio nei precedenti capitoli.

```
import pymysql
config = {
    'user': 'root',
    'password': '<password>',
    'host': '127.0.0.1',
    'database': 'biblioteca',
    'port': 3306
}
connessione = pymysql.connect(**config)
cursore = connessione.cursor()
cursore.execute("select * from paesi")
risultato = cursore.fetchall()
print(risultato)
connessione.close()
```

L'esecuzione dello script fornisce il seguente risultato:

```
((1, 'Italia'), (2, 'Gran Bretagna'), (3, 'USA'), (4, 'Russia'))
```

Se non ci piace questo modo grezzo di presentare il risultato e lo vogliamo esporre meglio possiamo intervenire inserendo, al posto dell'istruzione `print(risultato)`, il seguente blocchetto:

```
for i in range(len(risultato)):
    ri = risultato[i]
    ri1 = str(ri[0]) + ' ' + ri[1] + ' '
    print(ri1)
```

nel quale ogni elemento della lista risultato viene preso come una tupla isolata, ogni elemento della tupla viene accostato all'altro come stringa, con in mezzo un piccolo spazio libero, e ogni stringa risultante viene stampata.

Il risultato ora si presenta così:

```
1 Italia
2 Gran Bretagna
3 USA
4 Russia
```

Occorre ricordare che la stringa risultante da quest'ultima operazione è una stringa unicode. Pertanto, se essa, anziché sullo standard output, come avviene per default nel nostro esempio, fosse diretta a un file, la stringa `ri1` oggetto dell'istruzione `print` andrebbe scritta `ri1.encode('utf-8')`; in caso contrario avremmo un messaggio di errore e il file non potrebbe essere scritto.

5 MariaDB su server web

Nel Capitolo 1 ho chiarito che per creare e gestire un database con MariaDB non è necessario avere sul nostro computer un server web locale e nel seguito di questo Capitolo vedremo anche che ne possiamo fare benissimo a meno in quanto forse il gioco non vale la candela.

In realtà il server locale ci serve se dobbiamo costruire e collaudare in locale sul nostro computer un sito web vero e proprio che, una volta messo a punto e funzionante, andremo a trasferire presso un service provider per renderlo attivo sul web.

L'unica vera utilità, se ricorriamo all'installazione di un prodotto come XAMPP, di cui parlerò poco più avanti, è quella di avere a disposizione MariaDB affrontando la minore fatica possibile.

5.1 Installazione dell'occorrente

Per installare il server web locale possiamo installare una per una le componenti che ci servono e configurarle a dovere oppure possiamo ricorrere a prodotti già confezionati.

Installazione e configurazione diretta

Le componenti necessarie per avere un server locale su cui lavorare con MariaDB sono le seguenti.

Il server vero e proprio. Quello che va per la maggiore nel mondo del software libero e non solo è il server **Apache2**, disponibile per Linux, Windows e Mac.

Poi abbiamo bisogno del linguaggio di programmazione PHP per far funzionare il tool phpMyAdmin e questa esigenza viene soddisfatta con la versione CGI di PHP, il package **php-cgi**.

Ovviamente dobbiamo disporre di un **browser** web per provare l'esecuzione del programma.

Infine abbiamo bisogno di **MariaDB** e del tool **phpMyAdmin**.

I professionisti preferiscono installare tutte le cose che servono una per una, configurandole a dovere anche per renderle adatte alle proprie esigenze.

Si tratta però di un lavoro non da poco, specie per chi lavora su Windows o su Mac, e ai dilettanti cui mi rivolgo suggerisco la strada più facile che vedremo nel prossimo paragrafo.

Tuttavia, chi ha la fortuna di lavorare su MXLinux, una distro Linux che più bella non si può, dispone di uno strumento fenomenale, che si occupa di installare in blocco ciò che serve: basta andare su MXTOLS ▷ MX INSTALLA PROGRAMMI ▷ SERVER, selezionare LOCAL WEB SERVER e cliccare su INSTALLA.

In questo modo abbiamo una installazione in tutto simile a quella che avremmo avuto installando uno per uno gli strumenti necessari, già configurata e funzionante per normali esigenze, che vanno benissimo per un dilettante e non solo.

Su altre distro legate a Debian (Ubuntu, Mint, Mauna, ecc.) potremmo installare il server con l'installatore `tasksel`, ma funziona sì e no.

Può accadere che già abbiamo installato un server locale per utilizzare il linguaggio di programmazione PHP, come indicato nell'allegato in formato PDF «php» al mio articolo «Nel mondo di PHP» pubblicato sul mio blog all'indirizzo www.vittal.it lo scorso mese di aprile, nel qual caso abbiamo tutto quanto serve e non abbiamo bisogno di rifare le installazioni.

Ci rimane da installare il tool `phpmyadmin` per lavorare sul database.

Esistono alternative a questo tool, come MySQL Workbench, DBeaver, HeidiSQL, ma una vale l'altra e forse phpMyAdmin è ancora la migliore.

Prima di installare il tool è bene, per evitare successive complicazioni che ci farebbero perdere un sacco di tempo, creare un utente root con relativa password per il database MariaDB e lo facciamo scrivendo a terminale il comando

```
sudo mysql_secure_installation
```

che ci sottopone ad una serie di quesiti:

- . al primo di questi, Enter current password for root:

- se siamo alla prima installazione e non abbiamo ancora una password per l'utente root di MariaDB non scriviamo nulla e premiamo INVIO.

- . a tutti gli altri rispondiamo y,

- . quando ci viene richiesto indichiamo la password per l'utente root.

Una volta compiuto questo importantissimo passaggio installiamo il tool mancante con il comando a terminale

```
sudo apt install phpmyadmin
```

 se siamo su Debian e derivate,

`sudo dnf install phpmyadmin` se siamo su Red Hat e derivate.

Nel corso dell'installazione si apriranno alcune finestre di dialogo, la prima per chiederci quale server configurare automaticamente e rispondiamo scegliendo l'opzione relativa al server Apache2 e clicchiamo su NEXT.

La seconda per chiederci una password per il database, una prima volta per l'indicazione e una seconda volta per la conferma.

Proseguiamo sempre cliccando su NEXT fino a che l'installazione si conclude sottoponendoci una finestrella finale in cui scegliere il nome dell'utente, che indichiamo come root, e la password, per la quale indichiamo quella scelta durante la secure installation. Se non abbiamo particolari problemi di riservatezza ci conviene salvare queste indicazioni in modo da non doverle più ripetere.

Installazione di XAMPP

Chi non voglia avventurarsi nella procedura descritta nel precedente paragrafo, sia su sistema Linux diverso da MXLinux, sia su sistema Windows, sia su sistema Mac, ha la possibilità di utilizzare una suite software gratuita e open-source, denominata XAMPP, che permette di creare un server web locale sul proprio computer. Il suo nome è l'acronimo di X (che significa cross-platform, in quanto funziona su Linux, Windows e macOS), Apache, MariaDB, PHP e Perl (altro linguaggio di scripting che può essere utilizzato al posto di PHP).

All'indirizzo

<https://www.apachefriends.org/it/download.html>

possiamo scegliere la versione per il nostro sistema operativo e scaricarla.

Istruzioni per l'installazione e altro si trovano cliccando sul lato destro della finestra sui link alle domande frequenti per i vari sistemi operativi.

Ad installazione avvenuta, se non abbiamo scelto diversamente durante l'installazione, troviamo tutto il software che ci serve nelle seguenti posizioni del file system:

/opt/lampp per il sistema operativo Linux,

C:\XAMPP per il sistema operativo Windows,

/Application/XAMPP per il sistema operativo Mac.

5.2 phpMyAdmin

Una volta che abbiamo installato tutto quanto, per lavorare sul database dobbiamo innanzi tutto attivare il server.

Se abbiamo installato il software alla maniera normale, pezzo per pezzo, o aiutati dal tool di installazione di MXLinux che abbiamo visto nel precedente Paragrafo il server si attiva all'accensione del computer ed è sempre disponibile.

Se invece abbiamo installato XAMPP, quando abbiamo bisogno del server lo dobbiamo accendere e lo possiamo fare richiamando un'applicazione grafica in questo modo:

. se siamo su Linux, posizionati nella directory /opt/lampp, con il comando a terminale

```
sudo ./manager-linux-x64.run
```

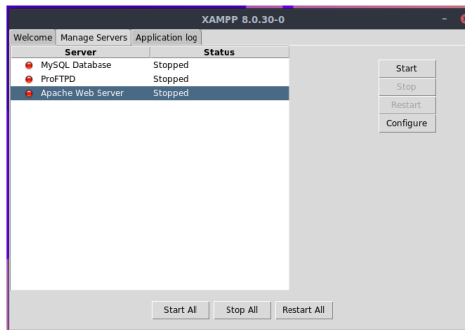
. se siamo su Windows, posizionati nella directory C:\xampp, con il comando a prompt dei comandi

```
xampp_control
```

. se siamo su Mac, posizionati nella directory /Applications/XAMPP, con il comando a terminale

```
manager-osx
```

La seguente illustrazione mostra l'applicazione grafica aperta su Linux, con selezionata la linguetta MANAGE SERVERS



Selezionate le voci MYSQL DATABASE e APACHE WEB SERVER, che sono quelle che qui interessano, clicchiamo sul pulsante START e il server si avvia.

Con qualche piccola differenza, abbiamo la stessa grafica sui sistemi operativi Windows e Mac.

In alternativa possiamo utilizzare i comandi a terminale, sempre posizionati nella directory di installazione di XAMPP:

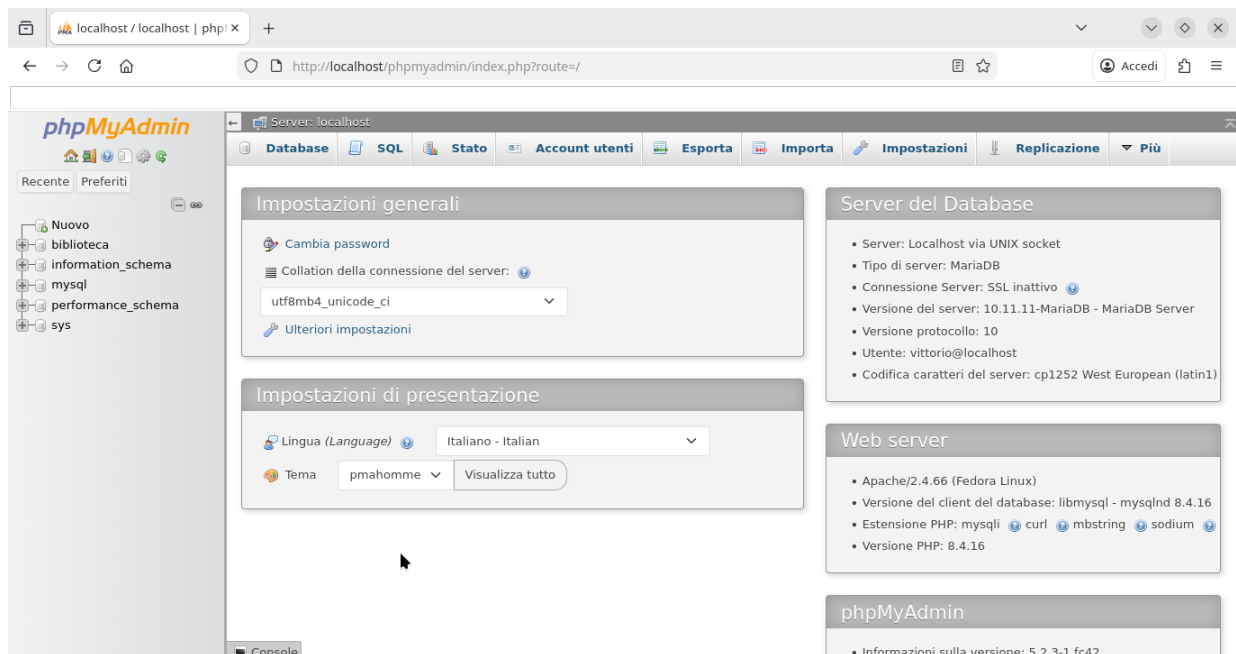
`xampp start` (in Linux `sudo ./xampp start`) per avviare il server,

`xampp stop` (in Linux `sudo ./xampp stop`) per disattivarlo (sapendo comunque che il server si disattiva allo spegnimento del computer).

A server avviato e se abbiamo installato tutto a dovere, inserendo nel nostro browser preferito l'indirizzo

<http://localhost/phpmyadmin/>

e superate le eventuali richieste di credenziali ci troviamo di fronte questa pagina web



Nella barra verticale sulla sinistra sono elencati i database esistenti.

Quelli denominati `information_schema`, `mysql`, `performance_schema` e `sys` sono i database al servizio del sito web che potremmo creare e collaudare nel nostro server locale prima di trasferirlo presso un service provider per renderlo attivo sul web.

Vediamo però che tra i database è elencato anche `biblioteca`, quello che è stato costruito con l'esercitazione svolta nei precedenti Capitoli.

Questo avviene in quanto sul computer su cui sto scrivendo questo manuale non ho installato XAMPP ma ho installato e configurato il server locale elemento per elemento.

In questo caso, dal momento che il database che abbiamo costruito si trova sul server di MariaDB (su Linux in `/var/lib/mysql/`) e dal momento che il server di MariaDB, al momento dell'installazione di Apache2, si integra con questo, phpMyAdmin lo trova.

Se lavorassimo su XAMPP i database ritrovabili da phpMyAdmin dovrebbero trovarsi nella sotto-directory `/var/mysql` della directory dove è installato XAMPP.

Per ritrovare il nostro database biblioteca su XAMPP dobbiamo esportarlo da MariaDB (con il comando `mariadb-dump`) e importarlo in phpMyAdmin nei modi che vedremo.

Verrebbe voglia di copiare semplicemente la directory contenente il database dal luogo in cui è stata salvata da MariaDB a riga di comando al luogo adatto per XAMPP ma non funziona, trattandosi di directory protetta da password.

Ma veniamo al funzionamento di phpMyAdmin.

Nella parte superiore della finestra abbiamo una barra di linguette (DATABASE, SQL, STATUS, ecc.) cliccando sulle quali apriamo altre finestre nelle quali, in maniera visuale, siamo guidati alle azioni da compiere.

L'unica finestra non di questo tipo è quella contrassegnata dal nome SQL, che si presenta semplicemente come un editor di testo in cui scrivere comandi in linguaggio SQL per ottenere ciò che vogliamo dopo aver premuto il pulsante ESEGUI che compare sotto la zona di editing. Come se stessimo lavorando nella shell di MariaDB e premessimo poi INVIO. Ovviamente in questa finestra possiamo fare tutto: dall'apertura di un nuovo database e di tabelle, alla loro alimentazione, alla loro modifica, alla produzione di un dump del database in un file `.sql`, ecc.

Per alcuni compiti siamo aiutati da finestre visuali.

La prima che vediamo è quella che ci aiuta a creare un nuovo database, che si apre cliccando sulla linguetta DATABASE o sulla voce NUOVO nella barra verticale di sinistra.

Inserito il nome del database da creare e cliccato il pulsante Crea ci viene presentata una nuova finestra per creare le tabelle, per ognuna delle quali ci vengono proposte finestre per crearne la struttura, cioè le colonne, ecc.

Nella finestra iniziale per la creazione di un nuovo database, sotto la zona in cui veniamo guidati ad indicarne il nome e a premere il pulsante per crearlo, abbiamo l'elenco dei database esistenti. In questo elenco possiamo selezionare database da eliminare o database su cui lavorare.

Selezionando la tabella del database su cui ci interessa lavorare nella barra verticale di sinistra o cliccando sul nome del database stesso nella finestra DATABASE e poi sul nome della tabella su cui ci interessa lavorare ci si presenta una finestra che mostra i dati contenuti nella tabella.

Nella barra delle linguette in alto possiamo cliccare sulla linguetta INSERISCI per ottenere una guida visuale all'inserimento di nuovi dati.

Il tutto molto ben guidato e di facile apprendimento.

Senza tuttavia raggiungere la perfezione delle maschere di inserimento che possiamo creare collegando il database a LibreOffice, come abbiamo visto si possa fare nel Capitolo 3.

Sempre nella finestra iniziale abbiamo le linguette ESPORTA e IMPORTA, che ci guidano rispettivamente alla produzione del file in formato SQL per esportare il database o alla sua importazione sul server.

Questo file, che ha estensione `.sql`, è semplicemente un file di testo che contiene tutte le istruzioni in linguaggio SQL per ricostruire il database in un altro ambiente di lavoro.

Personalmente non ritengo che gli aiuti forniti da phpMyAdmin siano gran che e preferisco di gran lunga la riga di comando e il collegamento a LibreOffice.

Se abbiamo installato il server componente per componente ci rimane aperta la strada a fare tutto questo.

Se abbiamo installato solo XAMPP per evitare il fastidio dell'installazione componente per componente abbiamo comunque modo di utilizzare la riga di comando e LibreOffice, come vediamo nei prossimi paragrafi.

5.3 MariaDB a riga di comando da XAMPP

Per lavorare a riga di comando avendo installato solo XAMPP abbiamo già visto che abbiamo a disposizione la finestra di editing di comandi SQL che si apre con la linguetta SQL della prima finestra su cui si apre il tool phpMyAdmin.

Ma possiamo anche aprire la shell di MariaDB utilizzando i seguenti comandi su terminale con il server XAMPP e database attivati.

su Linux

```
sudo /opt/lampp/bin/mariadb -u root
```

su Mac

```
sudo /Application/XAMPP/bin/mariadb -u root -p
```

La password da inserire è quella di amministratore Mac.

su Windows

Innanzitutto aprire XAMPP Control Panel, poi cliccare su SHELL e scrivere

```
mariadb -u root
```

Il database creato a riga di comando comparirà tra quelli riconosciuti da XAMPP e qui elaborabili.

5.4 MariaDB da XAMPP con LibreOffice Base

Si segue la stessa procedura descritta nel Capitolo 3 con l'unica differenza che, quando ci verrà chiesto il nome utente per verificare la connessione, dobbiamo indicarlo (sarà sempre molto probabilmente root) senza optare per l'opzione PASSWORD RICHIESTA e cliccare su VERIFICA LA CONNESSIONE.

Il server deve sempre essere attivo, sia quando creiamo la connessione sia quando lavoriamo sul database attraverso il file .odb di LibreOffice.

Parte III

Sqlite

La prima cosa che viene chiarita nella presentazione di SQLite che troviamo sul sito <https://sqlite.org/> è che SQLite non è paragonabile ai motori di database client/server come MySQL/MariaDB, Oracle, PostgreSQL, SQL Server e che non vuole competere con essi. SQLite, si dice, nasce per altri fini: primo tra tutti quello di essere un software molto leggero adatto per essere «embedded», cioè inserito in altri software all'interno dei quali può assolvere egregiamente la funzione di motore di gestione di database.

Tutto ciò significa che se vogliamo costruire un database destinato ad essere utilizzato contemporaneamente su più postazioni di lavoro, come può avvenire per il database dei correntisti di una banca o dei clienti di una multinazionale non possiamo certo ricorrere a SQLite. Ma se vogliamo costruire un database sul nostro personal computer, per i nostri usi personali o per la nostra piccola azienda, che possa essere utilizzato anche sul nostro tablet o sul nostro telefonino, SQLite è quello che fa per noi.

Il source code di SQLite è di pubblico dominio e può essere usato senza bisogno di licenza alcuna.

Esiste comunque un'organizzazione che se ne occupa, che rilascia licenze e assistenza a pagamento a chi utilizzi il software in costosi progetti complessi e che non voglia fondare questi progetti su un pubblico dominio che oggi c'è e domani non si sa. L'organizzazione si chiama Hwaci (pronuncia uàci) e, nonostante il nome, non è giapponese: sta per Hipp, Wyrick & Co., Inc. ed ha sede a Charlotte nel North Carolina. Hipp altro non è che Richard Hipp, l'inventore del software, la cui prima stesura risale al 2000, e Wyrick altro non è che la di lui moglie Ginger Wyrick.

Tutto in famiglia per una minuscola libreria software il cui kernel, scritto in linguaggio C, occupa poche centinaia di kB e che, con tutti gli accessori del caso, non occupa più di 5 mega sul nostro computer.

E questa piccola libreria si è rivelata il più efficiente motore di database che esista ed è diventato il database più diffuso al mondo.

Il limite principale lo abbiamo già sottolineato: non è un database client/server. Questo, tuttavia, può risultare un vantaggio, più che un limite. L'altro limite sta nel non riuscire a gestire database superiori a 2 TB (cioè 2 mila miliardi di byte), che penso, tuttavia, siano più che sufficienti per le esigenze di moltissima gente e di moltissime imprese.

I vantaggi vanno dal non essere un database client/server quando questa impostazione non serve e sarebbe ridondante per le nostre esigenze, al riunire in un unico file tutto il database, qualunque sia il numero di tabelle di cui è composto¹.

In questo manuale mi propongo di presentare alcuni modi di utilizzare SQLite, dando per scontato che il lettore sappia in che cosa consista un database, in particolare un database relazionale, spero aiutato da quanto ho detto nell'Introduzione.

1 Installazione

La versione corrente di SQLite si chiama **sqlite3**.

SQLite praticamente funziona su tutti i sistemi operativi; limitandomi a quelli più vicini a noi ricordo Linux, Windows, Mac OSX, Android, iOS e VSIX.

Per quanto riguarda apparecchiature equipaggiate Android non abbiamo nemmeno il disturbo di installarlo in quanto è già presente e così dovrebbe essere per tablet e smartphone equipaggiati

¹Occorre riconoscere che questi vantaggi li ritroviamo anche nel programma di gestione di database Access della Microsoft. Ma il paragone con Access è perdente, per Access, su tutti gli altri fronti: dal costo alla flessibilità di utilizzo.

con altri sistemi (iOS per quanto riguarda i prodotti Apple e VSIX per quanto riguarda Windows Phone).

Anche sui computer dovrebbe essere già installato insieme al sistema operativo. Per averne conferma basta lanciare il comando `sqlite3` e vedere cosa succede.

In ogni modo, dal sito <https://sqlite.org/> possiamo scaricare la versione adatta al nostro sistema operativo: l'ultima nata è la 3.15.1, rilasciata il 4 novembre appena scorso. Se usiamo Linux troviamo certamente la versione più adatta alla distro che abbiamo installato nel relativo repository.

Sullo stesso sito troviamo tutta la documentazione su SQLite, fatta molto bene ma solo in lingua inglese.

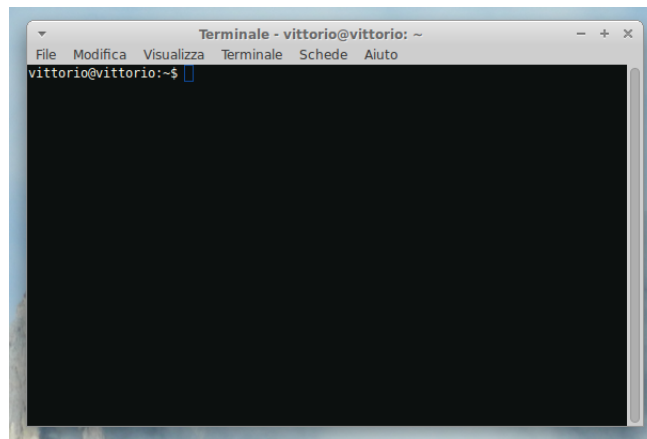
2 Uso di SQLite a riga di comando

Una volta installato, **sqlite3**, da solo, ci consente di creare e gestire un database senza bisogno di ricorrere ad altri programmi.

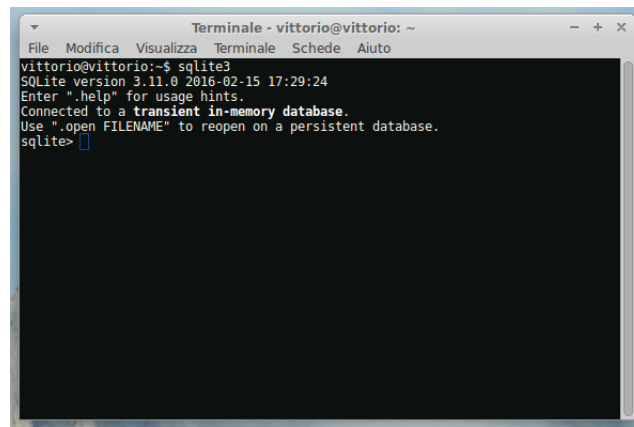
Essendo, come ho detto nel precedente paragrafo 1, un piccolissimo programma senza fronzoli e ridotto all'essenziale, se vogliamo usarlo da solo dobbiamo fare il sacrificio di scendere al suo basso livello e richiederne le prestazioni usando il linguaggio suo proprio attraverso il primordiale sistema dell'interfaccia a riga di comando.

L'interfaccia a riga di comando si basa su una interazione di tipo testuale tra utente ed elaboratore, che avviene attraverso l'uso della sola tastiera del computer per impartire istruzioni a questo, come si faceva un tempo, quando si dialogava con grossi mainframe, magari collocati in un luogo diverso da quello in cui lavorava l'utente, con apparecchiature prive di capacità elaborativa, chiamate terminali. Non a caso le interfacce a riga di comando che sono rimaste nei moderni computer si chiamano, pure loro, almeno per chi proviene dalla cultura informatica di quei tempi, terminali. E i programmi che ci propongono questo tipo di interfaccia sui moderni personal computer sono degli emulatori di terminale: in Linux e in Mac OS X, data la provenienza di entrambi i sistemi operativi dal sistema Unix, si chiamano **terminale** e ci vengono proposti nei menu delle applicazioni e in Windows si chiama **prompt dei comandi** e, ove non proposto nel menu, si può attivare con il comando `cmd`.

La seguente figura mostra il terminale del computer su cui sto lavorando, equipaggiato Linux



Per avviare il programma SQLite dobbiamo scrivere nella prima riga del terminale, appena dopo il simbolo `$` che vediamo nella figura, il comando `sqlite3` e qui vediamo il risultato

A screenshot of a terminal window titled "Terminale - vittorio@vittorio: ~". The window shows the command prompt where the user has entered 'sqlite3'. The output shows the SQLite version (3.11.0) and a message indicating it is connected to a transient in-memory database. The prompt 'sqlite>' is visible at the bottom of the terminal.

Nel terminale è comparso il prompt per i comandi SQLite, con cui veniamo avvertiti che SQLite è pronto (prompt) per ricevere comandi e scrivendo sulla riga che comincia con sqlite>

possiamo chiedere a SQLite di fare ciò che ci serve.

Per chiederlo dobbiamo utilizzare un linguaggio che SQLite comprenda: il linguaggio SQL (Structured Query Language), quello che tutti i programmi di gestione di database comprendono, integrato con alcuni comandi speciali tipici del programma SQLite, detti comandi puntati, in quanto vengono impartiti preceduti da un punto.

L'inserimento dei comandi può indifferentemente avvenire con lettere maiuscole o minuscole. I comandi si possono scrivere su più righe, andando a capo con INVIO, e terminano con il punto e virgola ;.

I comandi SQL riconosciuti da SQLite non sono proprio tutti ma i non riconosciuti sono quelli che praticamente non si usano mai².

L'elenco che segue riguarda i comandi di uso più frequente e ne indico la sintassi per le più comuni applicazioni.

Per ulteriori approfondimenti su questi e altri comandi rimando a uno dei numerosi manuali sul linguaggio SQL che si trovano in libreria o a ricerca su Internet.

CREATE TABLE, per creare una tabella

```
CREATE TABLE <nome_tabella> (<nome_colonna><tipo_dato>[attributi],  
                               <nome_colonna><tipo_dato>[attributi], ...);
```

gli attributi sono di inserimento facoltativo e i più frequentemente usati sono:

PRIMARY KEY, che rende il campo la chiave primaria della tabella (quella che serve per instaurare relazioni con altre tabelle);

NOT NULL, che rende il campo obbligatoriamente destinato a contenere un dato, contrariamente a quella che sarebbe l'impostazione di default;

AUTOINCREMENT, destinato solo a campi contenenti un numero per fare in modo che il contenuto del campo si incrementi automaticamente di una unità a partire dal più grande valore già presente³. Attributo molto utile per la primary key.

DROP TABLE, per cancellare una tabella

²Le funzionalità SQL non implementate in SQLite riguardano i comandi RIGHT e FULL OUTER JOIN (LEFT OUTER JOIN è implementato), GRANT e REVOKE. Non è implementata la funzione di scrittura nelle VIEWS (le VIEWS di SQLite sono in sola lettura). Per il comando ALTER TABLE sono implementate solo le funzioni RENAME TABLE e ADD COLUMN.

³Per chi conosce il linguaggio SQL chiarisco che, in luogo della scrittura AUTO_INCREMENT tipica di quel linguaggio, nell'uso con SQLite si deve usare la scrittura AUTOINCREMENT.

DROP TABLE <nome_tabella>;

INSERT, per inserire dati in una tabella

```
INSERT INTO <nome_tabella> (<colonna_interessata>, <colonna_interessata>, ...) VALUES (<valore>, <valore>, ...);
```

Ovviamente i valori vanno elencati nello stesso ordine delle colonne. I valori numerici si inseriscono tali e quali, mentre i valori espressi come testo vanno racchiusi tra semplici apici '.
SELECT, per leggere i dati dalle tabelle del database

```
SELECT <nome_colonna>, <nome_colonna>, ... FROM <tabella>, <tabella>, ... WHERE <condizioni>;
```

Questo è il comando più importante del linguaggio SQL in quanto è l'unico che viene o può convenientemente essere utilizzato anche quando lavoriamo con SQLite non a riga di comando. Per meglio fissare i concetti sulla sua sintassi sottolineo che

- dopo SELECT va inserito, attraverso il nome di colonna, l'elenco dei campi che si vogliono evidenziare nel risultato della ricerca (in gergo la query), separati da virgola e nell'ordine in cui si vogliono evidenziare; non appena vi è il minimo rischio di ambiguità e, comunque, nel caso i campi da evidenziare non corrispondano, nell'ordine o nel numero, a quelli che sono coinvolti nella ricerca, il campo va indicato non semplicemente come nome_colonna ma nella forma più completa tabella.nome_colonna;
- dopo FROM va inserito l'elenco delle tabelle coinvolte nella ricerca, coinvolte o per l'estrazione di campi da evidenziare o semplicemente perché contengono almeno un elemento utile alla ricerca;
- dopo WHERE, separate eventualmente da AND, si indicano le relazioni tra le tabelle coinvolte nella ricerca nella forma tabella.nome_colonna = tabella.nome_colonna e poi, sempre con AND, si aggiungono le condizioni di ricerca
 - nella forma tabella.nome_colonna like '.....' se abbiamo a che fare con dati espressi in forma di testo. Per esprimere la condizione like si possono usare i caratteri jolly che, nel linguaggio SQL sono:
 - * _ per un solo carattere,
 - * % per un numero imprecisato di caratteri;
 - nella forma tabella.nome_colonna = oppure > oppure < a un numero se abbiamo a che fare con dati numerici;
 - nella forma tabella.nome_colonna is <valore> se vogliamo esprimere un criterio di perfetta identità; <valore> può essere una stringa di testo (espressa tra apici), un numero o il valore null, scritto tale e quale senza apici oppure, sempre scritto senza apici, uno dei valori booleani true o false.

Sembra tutto molto complicato ma, dopo un po' di pratica, ci accorgiamo che non è così: alla fine del paragrafo proporrò un piccolo esempio che spero incoraggi in tal senso.

Nel comando per creare una tabella abbiamo visto che occorre indicare, per ciascuna colonna destinata a scandire i campi dei record del nostro database, il tipo del dato che vi può essere inserito. In SQLite i tipi di dato gestiti sono solo cinque:

integer, cioè numero intero

real, cioè numero reale
text, cioè testo
blob, cioè testo in forma binaria
null, cioè sconosciuto.

Dal momento che in altri contesti, per esempio nel motore di database MariaDB, i tipi sono molti di più, per rendere possibile l'importazione di dati da un database diverso da SQLite, sqlite3 riconosce delle affinità per incanalare sui propri tipi quelli di altri. Così verranno ascritti al tipo integer tutti i tipi altrove chiamati con un qualsiasi nome contenente la stringa int; verranno ascritti al tipo real tutti i tipi altrove chiamati con nomi contenenti la stringa floa o doub; verranno ascritti al tipo text tutti i tipi altrove chiamati con nomi contenenti la stringa char o clob.

Ci rimane da vedere quali sono i comandi tipici di sqlite3, i così detti comandi puntati.

Innanzitutto un chiarimento sul comando sqlite3 che utilizziamo per entrare nel prompt di sqlite3. Se lo scriviamo da solo apriamo la finestra terminale che ho riprodotto nella precedente figura 2, con il prompt. Se lo scriviamo seguito dal nome di un database, se il database esiste verrà aperto insieme al prompt, se il database non esiste verrà creato non appena avremo creato almeno una tabella. Tutto ciò sapendo che sqlite3 trova e memorizza i database nella directory dalla quale diamo il comando sqlite3. Pertanto, se vogliamo aprire un database esistente dobbiamo preventivamente recarci nella directory dove esso è memorizzato.

Una volta entrati nel prompt di sqlite3, con il comando puntato **.help** possiamo vedere l'elenco di tutti i comandi puntati disponibili con la descrizione di ciò che fanno.

Qui riporto quelli di uso più comune.

.exit oppure **.quit** per uscire dal prompt di sqlite3

.schema [<tabella>] mostra lo statement SQL di creazione di tutto il database o della tabella, se indicata

.import <file> <tabella> importa i dati del file indicato nella tabella indicata. Occorre preventivamente indicare a sqlite3 il simbolo separatore che si trova nel file di dati con il comando **.separator** '<simbolo>': il simbolo potrà essere la virgola ,, il punto e virgola ;, il carattere |, ecc.

.mode <modo> imposta il formato di output.

<modo> sta per	csv	valori separati da virgola
	column	valori in colonna allineati a sinistra
	line	un valore per riga
	tabs	valori separati da uno spazio di tabulazione
	tcl	valori come lista di elementi TCL
	list	valori separati dal simbolo scelto con .separator

.output <file> reindirige l'output sul file indicato, creandolo se non c'è

.output stdout reindirige l'output allo schermo (impostazione di default)

.dump salva il database in formato SQL. Per salvare in un file occorre preventivamente reindirigare l'output con il comando **.output** <file>. Se si fa seguire al comando **.dump** il nome di una tabella, verrà salvata solo la tabella.

.read <file> ripristina il database leggendo il file in formato SQL creato con il comando **.dump**.

Un esercizio istruttivo

Supponiamo di avere una biblioteca di quattro libri e, anche se totalmente inutile per padroneggiarla ma molto utile come esercizio in questa sede, vogliamo costruire un database per gestirla.

I libri sono

I promessi sposi di Alessandro Manzoni
Un luogo chiamato libertà di Ken Follett
I pilastri della terra di Ken Follett
Furore di John Steinbeck.

Il primo passo, sicuramente il più importante, è la progettazione del database e, per affrontarlo, dobbiamo innanzi tutto chiederci a cosa vogliamo che serva questo database. Nel nostro caso supponiamo che le esigenze siano: sapere le opere che abbiamo dei vari autori, sapere le opere che abbiamo scritte in certi periodi, sapere gli autori che abbiamo rappresentati nella nostra biblioteca per Paesi di appartenenza, sapere quali opere abbiamo di autori di un certo Paese.

Da qui deriviamo che i dati che dovremo inserire nel database saranno: i nomi degli autori, i titoli delle opere, le date di nascita e di morte degli autori, i Paesi di appartenenza degli autori.

Dal momento che la regola d'oro del buon database è «scrivere le cose una ed una sola volta», sia pure per colpa del solo Ken Follett, di cui abbiamo due libri, non possiamo ricorrere ad una tabella gestita in un foglio di calcolo in quanto dovremmo scrivere due volte il nome di Ken Follett, due volte la sua data di nascita, due volte il nome del Paese di appartenenza. Dobbiamo allora ricorrere ad un database relazionale e SQLite è qui per questo.

Una organizzazione ottimale del database che assolvere alle finalità che ci siamo posti potrebbe essere quella di impostare il database su tre tabelle: una per gli autori, contenente le date di nascita e di morte, una per i Paesi di appartenenza e una per i titoli delle opere. La tabella degli autori si dovrà relazionare con quella dei Paesi e con quella dei titoli delle opere e quest'ultima basterà che si relazioni con quella degli autori, attraverso la quale finirà per essere relazionata anche con la tabella dei Paesi.

La tabella autori dovrà pertanto avere cinque colonne: idAutore, che assolverà la funzione di chiave primaria e conterrà il numero di riferimento corrispondente all'autore, nome, che conterrà il nome dell'autore, nascita, che conterrà la data di nascita dell'autore, morte, che conterrà la data di morte dell'autore e idPaese, che conterrà il numero di riferimento del Paese derivante dalla tabella dei Paesi.

La tabella paesi avrà due colonne: idPaese, che assolverà la funzione di chiave primaria e conterrà il numero di riferimento corrispondente al Paese e paese, contenente il nome del Paese.

La tabella titoli avrà tre colonne: idTitolo, che assolverà la funzione di chiave primaria e conterrà il numero di riferimento corrispondente al titolo di ciascuna opera, titolo, che conterrà il titolo dell'opera e idAutore, che conterrà il numero di riferimento corrispondente all'autore derivante dalla tabella autori.

La progettazione è finita ed ora dobbiamo passare alla realizzazione.

Apriamo un terminale (se non troviamo altrimenti il modo, in Windows, lanciando il comando cmd), se vogliamo ci portiamo in una directory comoda per l'archiviazione del nostro database (in tutti i sistemi operativi, per fare questo, si scrive il comando cd seguito dall'indirizzo della directory in cui vogliamo spostarci) e, volendo dare al nostro database il nome «biblioteca», scriviamo il comando

```
sqlite3 biblioteca
```

aprendo così il prompt di sqlite3, che apparirà come in figura 2, già pronto anche per memorizzare il database che costruiremo in un file chiamato «biblioteca». Da qui in poi scriveremo tutti i nostri comandi nella prima riga libera aperta che inizia con il prompt sqlite>.

Ora creiamo la prima delle tre tabelle che ci servono, che chiamiamo «autori», con il comando create table autori (idAutore integer primary key autoincrement, nome text, nascita integer, morte integer, idPaese integer);

Poi creiamo la tabella dei Paesi, che chiamiamo «paesi», con il comando: create table paesi (idPaese integer primary key autoincrement, paese text);

Infine creiamo la tabella delle opere, che chiamiamo «titoli», con il comando create table titoli (idTitolo integer primary key autoincrement, titolo text, idAutore integer);

sempre non dimenticando di chiudere i comandi con il punto e virgola.

Se facciamo qualche errore o scriviamo parole che sqlite3 non comprende sarà sua cura avvisarci; se vogliamo comunque verificare che tutto sia andato bene possiamo scrivere il comando puntato .schema e nel terminale vedremo elencati i nostri tre comandi, a conferma che sqlite3 li ha capiti ed acquisiti. Da questo momento, nella directory in cui stiamo lavorando, compare il file «biblioteca».

Non ci resta che inserire i dati. Questa è la cosa più brutta da fare lavorando da terminale e nei paragrafi successivi vedremo sistemi più comodi per farlo. Ma questo esercizio serve per capire come lavora sqlite3 ed ha la sua utilità.

Cominciamo ad inserire i Paesi, visto che la chiave del Paese (idPaese) ci servirà per alimentare la tabella degli autori, e lo facciamo con i comandi

```
insert into paesi (paese) values ('Italia');
```

```
insert into paesi (paese) values ('Gran Bretagna');
```

```
insert into paesi (paese) values ('USA');
```

Abbiamo inserito i paesi, rispettivamente, di Alessandro Manzoni, di Ken Follett e di John Steinbeck, senza inserire il dato della prima colonna, idPaese, che avevamo definito autoincrementale.

Se scriviamo il comando

```
select idPaese, paese from paesi;
```

chiedendo a sqlite3 di selezionare i dati contenuti nelle colonne idPaese e paese della tabella paesi, otteniamo la risposta

```
sqlite> select idPaese, paese from paesi;
1|Italia
2|Gran Bretagna
3|USA
sqlite> 
```

a dimostrazione del fatto che sqlite3 ha attribuito automaticamente i valori 1, 2 e 3, nell'ordine, a Italia, Gran Bretagna e USA.

Inseriamo ora gli autori, con i comandi

```
insert into autori (nome, nascita, morte, idPaese) values ('Manzoni, Alessandro', 1785, 1873, 1);
```

```
insert into autori (nome, nascita, morte, idPaese) values ('Follett, Ken', 1949, null, 2);
```

```
insert into autori (nome, nascita, morte, idPaese) values ('Steinbeck, John', 1902, 1968, 3);
```

sempre trascurando l'inserimento del dato nella prima colonna, quella dell'idAutore, in quanto la fa sqlite3, dando automaticamente l'identificativo 1 a Manzoni, 2 a Follett e 3 a Steinbeck, nell'ordine con cui li abbiamo inseriti. Dal momento che Ken Follett è vivente, in luogo della data di morte abbiamo inserito il valore null. Nella colonna idPaese abbiamo inserito gli identificativi di Paese desunti dalla relativa tabella. Notare che, mentre i valori numerici si inseriscono tali e quali, i testi (stringhe) vanno racchiusi tra apici.

Finalmente tocca alle opere, che inseriamo con i comandi

```
insert into titoli (titolo, idAutore) values ('I promessi sposi', 1);
```

```
insert into titoli (titolo, idAutore) values ('Un luogo chiamato libertà', 2);
```

```
insert into titoli (titolo, idAutore) values ('I pilastri della terra', 2);
```

```
insert into titoli (titolo, idAutore) values ('Furore', 3);
```

ed abbiamo così creato il database della nostra biblioteca.

A questo punto vediamo come possiamo utilizzarlo per avere alcune informazioni sulla nostra raccolta letteraria.

Per conoscere le opere scritte da Ken Follett, in questo spezzone di schermata di terminale

```
sqlite> select titolo from titoli, autori where titoli.idAutore = autori.idAutore
and autori.nome like 'Follett%';
Un luogo chiamato libertà
I pilastri della terra
```

vediamo la query e il risultato. Nella query, in linguaggio SQL, chiediamo di selezionare il titolo delle opere ricorrendo alle tabelle titoli e autori, relazionate tra loro dall'identificativo dell'autore presente in entrambe e in corrispondenza di un autore il cui nome comincia con Follett. Puntualmente sqlite3 ci restituisce l'elenco delle opere di Ken Follett che abbiamo in biblioteca.

Per conoscere le opere scritte da autori americani, in questo spezzone di schermata di terminale

```
sqlite> select nome, titolo from autori, titoli, paesi where titoli.idAutore = a  
utori.idAutore and autori.idPaese = paesi.idPaese and paesi.paese like 'USA';  
Steinbeck, John|Furore
```

abbiamo query e risultato. Nella query, in linguaggio SQL, chiediamo di selezionare autore e titolo delle opere, ricorrendo alle tabelle autori, titoli e paesi, autori e titoli relazionate tra loro dall'identificativo dell'autore presente in entrambe e autori e paesi relazionate tra loro dall'identificativo del Paese presente in entrambe, in corrispondenza di un paese che si chiama USA. E sqlite3 ci indica l'unica opera scritta da un autore americano che abbiamo in biblioteca, come richiesto, con il nome dell'autore e il titolo.

Nel caso che segue abbiamo chiesto autore e titolo di opere scritte da autori nati prima del 1900

```
sqlite> select nome, titolo from autori, titoli where autori.idAutore = titoli.i  
dAutore and autori.nascita < 1900;  
Manzoni, Alessandro|I promessi sposi
```

ed abbiamo puntualmente ottenuto la risposta.

Ora abbiamo voluto conoscere autore e titolo di opere scritte da autori viventi

```
sqlite> select nome, titolo from autori, titoli where autori.idAutore = titoli.i  
dAutore and autori.morte is null;  
Follett, Ken|Un luogo chiamato libertà  
Follett, Ken|I pilastri della terra
```

ed otteniamo l'elenco delle opere di Follett, l'unico vivente che abbiamo in biblioteca.

Finora abbiamo ottenuto risposte sullo schermo.

Se volessimo ottenere risposte su supporto meno volatile, o per conservarle o per stamparle, potremmo far precedere al comando select il comando .output <nome_file> come fatto in questo esempio

```
sqlite> .output opere_di_Ken_Follett  
sqlite> select titolo from titoli, autori where titoli.idAutore = autori.idAutor  
e and autori.nome like 'Follett%';  
sqlite> █
```

dove vediamo che sullo schermo non compare alcuna risposta ma, se andiamo a vedere nella directory in cui stiamo lavorando, troviamo un file di testo denominato, come richiesto, «opere_di_Ken_Follett», con l'elenco delle opere di Ken Follett che abbiamo in biblioteca.

Attenzione che il comando .output <nome_file> rimane valido per tutte le query che facciamo nel seguito della corrente sessione di lavoro e tutti i risultati che otterremo andranno ad accodarsi al testo contenuto nel file <nome_file>, fino a quando, con un altro comando .output <altro_nome_file> reindirizzeremo su un altro file i risultati delle nostre ricerche, o, con un comando .output stdout li faremo ritornare sullo schermo.

3 Interfacce grafiche per SQLite

Nel paragrafo precedente abbiamo esaminato il modo più ruspante di usare SQLite, quello che forse preferiscono gli smanettoni. Da mezzo smanettone quale sono, pur apprezzando la semplicità

e la scorrevolezza dell'uso della riga di comando per impostare le tabelle e le query, devo riconoscere che l'inserimento dei dati dalla riga di comando non è il massimo, soprattutto se la biblioteca contiene alcune migliaia di libri e non quattro, come nell'esercizio appena svolto.

Il mondo del software libero ci propone qualche soluzione semplificante, con piccoli programmi abbastanza leggeri e semplici da usare che ci danno modo di utilizzare SQLite senza conoscere i comandi puntati e riducendo la necessità di conoscere il linguaggio SQL al solo comando select.

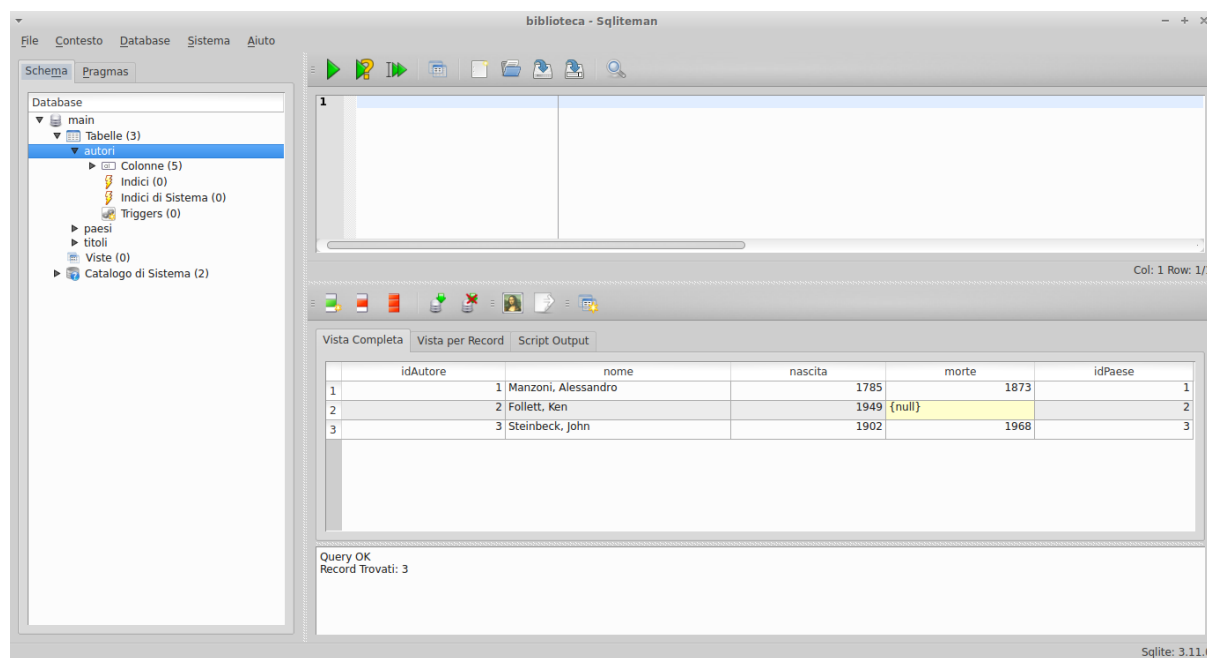
La scelta del programma che presento in questa sede è stata ispirata dal fatto che si tratta di un programma che si comporta egregiamente su tutti i sistemi operativi Linux, Mac OS X e Windows, che ha un'interfaccia in italiano (pur avendo, purtroppo, l'help in inglese) e che, a parte innumerevoli funzionalità anche per me ancora misteriose, è semplicissimo da usare: si chiama **Sqliteman**. Il suo difetto è di essere un po' antiquato e di non avere avuto aggiornamenti da alcuni anni.

Girovagando su Internet possiamo trovare altre cose dello stesso tipo: da **SQLitebrowser**, pure disponibile per tutti i tre sistemi operativi, a **SQLiteStudio**, disponibile solo per Linux.

Per Android molto bella la app **aSQLiteManager**, che troviamo su Google Play e che ha un funzionamento molto simile a Sqliteman che adesso vedremo.

Sqliteman possiamo scaricarlo dal sito <http://sqliteman.yarpen.cz/>. Chi usa Linux lo può installare dal repository della distro.

Qui mostro la schermata di lavoro di Sqliteman.



Vi è caricato il database che abbiamo creato nell'esercizio svolto nel paragrafo precedente e, in particolare, è visibile la tabella autori di quel database.

La finestra verticale a sinistra è dedicata alla navigazione nelle tabelle del database che avviene in modo assolutamente intuitivo. Con doppio click sul nome della tabella rendiamo visibile la tabella stessa nella finestra centrale sulla destra.

La finestra sulla destra in alto è dedicata all'inserimento dei comandi in linguaggio SQL, se desideriamo usarli o se dobbiamo usarli, come nel caso delle ricerche.

Nella finestra sulla destra in basso troviamo i messaggi che SQLite ritiene di inviarci, tipo quelli di OK se tutto va bene o di errore se qualche cosa va male.

Una passeggiata tra le voci del menu ci fa capire immediatamente cosa possiamo fare con questo programma (affinché le voci del menu siano tutte apribili è necessario sia caricato un database) e il come farlo è assolutamente intuitivo.

Per creare un nuovo database scegliamo FILE ▸ NUOVO e indichiamo dove salvare e come nominare il database; per creare le tabelle scegliamo DATABASE ▸ CREA TABELLA. Dopo aver letto il precedente paragrafo non dovremmo avere alcun dubbio su come proseguire.

L'inserimento dei dati può avvenire in due modi:

- utilizzando i comandi in linguaggio SQL scrivendoli nella finestra sulla destra in alto e attivandoli cliccando, man mano per ognuno di essi, sul pulsante  (possiamo anche scrivere più comandi e attivarli in blocco, posizionandoci sul primo da inserire e cliccando sul pulsante );
- lavorando nella finestra centrale di destra, previa apertura della tabella in cui vogliamo inserire i dati. L'inserimento comincia cliccando sul pulsante  (nuova riga): ciò produce una nuova riga vuota nella tabella e, con doppio click sulle varie zone corrispondenti alle varie colonne, apriamo una finestrella di input in cui scrivere il dato da inserire. Una volta scritti tutti i dati per tutte le nuove righe diamo il via all'inserimento nel database cliccando sul pulsante .

Nel primo caso abbiamo tutta la scomodità che avevamo riscontrato nell'inserimento da riga di comando visto nel paragrafo precedente. Nel secondo è un po' meglio, ma non troppo.

Per quanto riguarda le ricerche si deve ricorrere al linguaggio SQL, scrivendo le query nella finestra in alto a destra e eseguendole con click sul pulsante . Il risultato compare nella finestra centrale di destra. Se scegliamo dal menu DATABASE ▸ CREA QUERY ci troviamo proposto un sistema alternativo praticamente inservibile se la ricerca coinvolge più di una tabella.

In poche parole, questi programmini che forniscono una interfaccia grafica per lavorare con SQLite danno un tocco di modernità all'approccio ma non è che semplifichino gran che.

4 Uso di SQLite con LibreOffice Base

Un modo egregio che ci offre il mondo del software libero per lavorare comodamente con SQLite è di farlo attraverso il modulo Base di LibreOffice. In questo modo, una volta creato il database e impostate le sue tabelle con il software sqlite3 (direttamente da terminale secondo quanto abbiamo visto nel Capitolo 2) o ricorrendo ad una interfaccia grafica secondo quanto abbiamo visto nel Capitolo 3) lo colleghiamo al modulo Base e, da qui in poi, alimentazione del database, ricerche e quant'altro lo facciamo con tutte le potenti funzionalità che ci offre Base: a partire dalla possibilità di alimentare il database con comode maschere di inserimento fino a quella di produrre risultati di ricerche ben impaginati e inseribili in relazioni dall'aspetto professionale.

Tutto ciò che troviamo descritto nella Parte I di questo manuale.

Ci si può chiedere: ma, dal momento che il modulo Base può lui stesso creare un database, che bisogno c'è di dargli in pasto un database di altra natura come SQLite?

La risposta è semplice: il database creato da Base è utilizzabile solo con Base mentre il database creato con sqlite3 è utilizzabile ovunque. Basta copiare il piccolo file che lo contiene aggiornato nei luoghi dove lo vogliamo utilizzare: altra directory del computer, chiavetta trasportabile, tablet o smartphone (che abbiamo visto essere naturalmente dotati di sqlite3).

L'ottenimento di tutti questi benefici costa ovviamente qualche fatica, che ritengo comunque sia ben compensata dai risultati: prima fra tutte la fatica di predisporre il computer ad instaurare una connessione tra database e altre applicazioni.

Open DataBase Connectivity (ODBC)

Il sistema ODBC, almeno nel caso di SQLite, ritengo sia relativamente il più semplice ed il più stabile per creare una connettività di database con altre applicazioni: l'alternativa sarebbe la Java

DataBase Connectivity (JDBC) che mi sembra più complicata e ritengo più adatta, eventualmente, per database MySQL.

Per la creazione di un contesto di connettività i problemi da superare sono due:

- sul computer devono essere installati i driver per il collegamento ad un certo tipo di database, nel nostro caso SQLite,
- il computer deve darci la possibilità di creare ed evidenziare l'origine dei dati (Data Source Name, DSN) per instaurare il collegamento.

Sistema operativo Linux Ubuntu e derivate, compreso Linux Mint

Se abbiamo la fortuna di usare uno di questi sistemi operativi entrambi i problemi si risolvono impartendo da terminale, con collegamento Internet attivo, il seguente comando `sudo apt-get install libsqliteodbc unixodbc`, nelle versioni più recenti semplificabile in `sudo apt install libsqliteodbc unixodbc`.

Con questo comando installiamo il driver (package **libsqliteodbc**) e instauriamo la possibilità di indicare i DSN. Ad installazione avvenuta, grazie al package **unixodbc**, nella directory `/etc` troviamo i due file

`odbcinst.ini`,
`odbc.ini`,

che sono la chiave di tutto.

Entrambi in formato testo, il primo (`odbcinst.ini`) ha il seguente contenuto derivante dal package `libsqliteodbc` e che teniamo tale e quale:

```
[SQLite]
Description=SQLite ODBC Driver
Driver=libsqliteodbc.so
Setup=libsqliteodbc.so
UsageCount=2
```

```
[SQLite3]
Description=SQLite3 ODBC Driver
Driver=libsqlite3odbc.so
Setup=libsqlite3odbc.so
UsageCount=2
```

ove si dà conto dei driver installati.

Nel secondo (`odbc.ini`) dobbiamo inserire i nomi e gli indirizzi dei database che desideriamo diventino delle origini dati (DSN), nel caso del nostro database «biblioteca» che io ho archiviato in una directory Database della mia home:

```
[biblioteca]
Driver = SQLite3
Database = /home/vittorio/Database/biblioteca
```

In questo modo abbiamo creato l'origine dati `[biblioteca]` abbinando ad essa il driver `SQLite3` e l'indirizzo del database.

Per poter memorizzare quanto scriviamo ricordiamo di aprire come superuser il file nell'editor di testo che useremo. Se, per esempio, usiamo GEDIT dovremo aprire il file con il comando `sudo gedit /etc/odbc.ini`.

Quanto possiamo fare semplicemente con l'editor di testo potremmo anche farlo con interfaccia grafica installando il package **unixODBC-bin**. La ritengo tuttavia una strada inutilmente complicata e non la consiglio.

Linux in generale

Quanto detto per Ubuntu e derivate vale ovviamente per tutti i vari sistemi Linux in circolazione, fatti salvi gli eventuali differenti comandi per installare i packages `libsqliteodbc` e `unixodbc`.

Sistema operativo Mac OS X

All'indirizzo <http://www.ch-werner.de/sqliteodbc/> possiamo trovare i driver `odbc` per `sqlite3` destinati al sistema operativo Mac OS X nel file **sqlite3-odbc-x.xx.dmg**, azionando il quale effettuiamo l'installazione.

Fratello di Linux per la comune derivazione da Unix, anche Mac OS X lavora con `unixodbc`, già installato insieme ad una utilità di interfaccia grafica **ODBCManager**, attraverso la quale possiamo creare i DSN.

Come in Linux, i file `odbcinst.ini` e `odbc.ini`, che si trovano nella directory `/Library/ODBC`, possono essere scritti e modificati con un editor di testo.

Sistema operativo Windows

Sempre all'indirizzo <http://www.ch-werner.de/sqliteodbc/> troviamo i driver `odbc` per `sqlite3` destinati al sistema operativo Windows nei file installatori **sqliteodbc.exe** e **sqliteodbc_w64.exe**, rispettivamente per sistemi a 32 o a 64 bit.

Una volta installati i driver dobbiamo per forza ricorrere ad un'applicazione per nominare i DSN: in Windows il file `odbc.ini` non è così a portata di mano e di editor di testo come avviene per i sistemi Linux e OS X in quanto si trova in una sottochiave del Registro di Sistema che, in Windows, è il *sancta sanctorum* che è meglio il poco esperto non tocchi mai, pena il rischio di dover buttare tutto.

L'applicazione fa parte del sistema operativo e si chiama **Amministrazione origini dati ODBC**: se non la troviamo nei menu la possiamo far partire con il comando `odbcad32` (l'eseguibile si chiama, infatti, `odbcad32.exe`). Funziona in maniera del tutto analoga all'**ODBCManager** del Mac. Se, una volta lanciato, apriamo la scheda **DRIVER** vediamo l'elenco di tutti i driver che abbiamo installati e, se li abbiamo installati con uno degli eseguibili sopra indicati, l'elenco comprenderà **SQLite ODBC Driver** e **SQLite3 ODBC Driver**.

Per creare i DSN apriamo la scheda **DSN UTENTE** e clicchiamo sul pulsante **AGGIUNGI**.

Dall'elenco dei driver che compare nella finestra successiva selezioniamo **SQLITE3 ODBC DRIVER** e clicchiamo sul pulsante **FINE**.

Si apre così una nuova finestra intitolata **SQLITE3 ODBC DSN CONFIGURATION**. Nella prima zona di immissione, intitolata **DATA SOURCE NAME**, scriviamo il nome che vogliamo dare alla nostra sorgente dati (se vogliamo riferirci al database «biblioteca» del precedente esercizio scriviamo *biblioteca*). Nella successiva zona di immissione, intitolata **DATABASE NAME**, inseriamo l'indirizzo del database, aiutandoci con il pulsante **BROWSE**. Il resto possiamo fare a meno di inserirlo.

Con **OK** su questa finestra e sulla successiva usciamo dal programma.

Collegiamo il database

Lanciamo LibreOffice Base e, nella finestra che si apre, intitolata **CREAZIONE GUIDATA DATABASE**, scegliamo di procedere con **COLLEGA A UN DATABASE ESISTENTE**. Clicchiamo in fondo a destra della finestrella, scegliamo **ODBC** dall'elenco che ci viene proposto e clicchiamo su **AVANTI**.

Nella successiva finestra, inseriamo l'origine dati (il DSN) nella zona di inserimento intitolata **NOME DELLA SORGENTE DATI ODBC NEL SISTEMA**, andando a sceglierlo nell'elenco che compare

cliccando sul pulsante SFOGLIA. Se abbiamo fatto bene tutto quanto previsto nel capitoletto precedente, l'elenco contiene il nome «biblioteca», lo selezioniamo, clicchiamo su OK in modo che il nome si veda nella zona di inserimento e clicchiamo su AVANTI.

Passiamo senza fare nulla oltre la prossima finestra, dedicata alla configurazione di un eventuale sistema di autenticazione, che qui non ci interessa, premendo su AVANTI.

Arriviamo così alla finestra finale dove ci viene proposto di registrare o meno il database in LibreOffice (consiglio di scegliere SÌ e dirò subito il perché). Possiamo anche scegliere se aprire subito il database o no. Cliccando su FINE creiamo un database di LibreOffice Base, che sarà un file con estensione .odb e che sceglieremo dove memorizzare e con quale nome nella successiva finestra (nel nostro caso consiglio il nome biblioteca nella stessa directory dove abbiamo il file sqlite3).

La registrazione del database in LibreOffice consente di collegare il database non solo al modulo Base ma anche agli altri moduli di LibreOffice, con utilità che vedremo nel seguito. Se non lo facciamo subito abbiamo comunque modo di registrare il database anche in un momento successivo.

Se abbiamo operato come suggerito, ora troveremo memorizzati due file biblioteca: uno, senza estensione, è il file del database sqlite3 (alcuni suggeriscono di attribuirgli l'estensione .db per sapere di che cosa si tratta) e l'altro, con estensione .odb, è l'impalcatura che gli creiamo attorno con LibreOffice.

Aperto il file .odb così creato possiamo lavorarlo secondo quanto abbiamo visto nella Parte I di questo manuale, come fossimo in presenza di un database creato con Base e tutto ciò che faremo avrà effetto sul collegato database sqlite.

5 Collegamenti attraverso i principali linguaggi di programmazione.

All'indirizzo <http://www.w3ii.com/it/sqlite/default.html>, oltre che trovare una buona guida a SQLite in generale, possiamo vedere svolte esercitazioni di collegamento a database sqlite3 utilizzando i linguaggi C/C++, Java, PHP, Perl e Python, collegamenti destinati ad interagire con il motore sqlite3 per creare database, tabelle, alimentarle, leggerle.

La guida è nata in lingua inglese e quella che consultiamo all'indirizzo sopra indicato ne è una traduzione evidentemente fatta da un traduttore non umano e contiene qualche strafalcione. Per chi conosce l'inglese è comunque accessibile la versione originaria scegliendo la lingua inglese nella finestrella di scelta in alto a destra della pagina web.

Lasciamo C e C++ ai professionisti in quanto di quelli sopra citati è il più ostico, a partire dalla necessità di procurarsi l'header sqlite3.h, sapere dove metterlo, ecc. Per chi voglia provare a svolgere qualche esercizio suggerito nella citata guida in C/C++ segnalo la via diretta che ci fornisce Linux Ubuntu per sistemare il compilatore GNU: basta installare la libreria di sviluppo con il comando `sudo apt-get install libsqlite3-dev`. Trattandosi di libreria esterna, bisogna ricordare, oltre all'inclusione dell'header sqlite3.h, di linkare la libreria al momento della compilazione con l'opzione `-l sqlite3`.

Anche per Java l'utilizzo non è direttamente possibile: occorre un binding con il file `sqlite-jdbc.jar` che si può trovare su Internet. Per chi usa Netbeans è installabile come plugin.

Quanto a PHP, tradizionalmente legato a MySQL, da quando sono sorti alcuni problemi di licenza l'implementazione nativa del linguaggio non è più quella di MySQL ma è proprio quella di SQLite, attraverso la libreria PDO, integrata all'interno di PHP.

Perl non è direttamente utilizzabile ma va arricchito del modulo Perl DBI, che possiamo procurarci collegandoci al sito <https://dbi.perl.org/>, dove si trova pure la documentazione. L'uso di Perl non è facile.

Fin da Python 2.5, cioè da anni, il modulo SQLite è incluso nella libreria standard, inizialmente `pysqlite2` e, da Python 2.6, `sqlite3`. Ritengo che l'approccio di Python per lavorare con SQLite sia il

più semplice di tutti e ho pensato di illustrarlo brevemente, riferendomi alla versione 3 di Python, che è ormai quella corrente e, ovviamente, a sqlite3.

Utilizzo di Python3

Una volta importato il modulo con

```
import sqlite3
```

creiamo una connessione con il database, che chiamiamo con un nome evocativo, per esempio, **db**, con

```
db = sqlite3.connect("<database>")
```

dove <database> sta per percorso e nome del database, messi tra doppi apici come stringa.

Poi, per lavorare sul database con cui siamo collegati, utilizzando il metodo `cursor()` della connessione creiamo un cursore, che chiamiamo, per esempio, con la sua iniziale **c**, con

```
c = db.cursor().
```

I principali metodi del cursore così creato ci consentono di eseguire sul database comandi SQL o di leggere i risultati di una query, in particolare:

`c.execute("<comando_SQL>")` esegue un comando SQL indicato come stringa o come variabile stringa;

`risultato = c.fetchall()` crea una lista, chiamata risultato, nella quale ogni elemento è una tupla che rappresenta la riga di una astratta tabella con i dati della query per riga e colonna, come estratti dal database su cui è stata fatta la ricerca, dati a loro volta rappresentati sotto forma di stringhe unicode.

Alla fine del nostro lavoro dobbiamo chiudere il database con

```
db.close().
```

In breve penso di aver detto le cose che servono nella stragrande maggioranza dei casi. Una documentazione completa sulla semplicità e la potenza di Python per fare queste cose si trova all'indirizzo <https://docs.python.org/2/library/sqlite3.html>.

Facciamo un piccolo esempio: ci proponiamo di estrarre il contenuto della tabella dei Paesi del database «biblioteca» che abbiamo creato e alimentato per esercizio nei precedenti capitoli.

Lo script Python, secondo quanto abbiamo appena visto, sarà il seguente:

```
import sqlite3
db = sqlite3.connect("/home/vittorio/Database/biblioteca") 4
c = db.cursor()
c.execute("select * from paesi")
risultato = c.fetchall()
print(risultato)
db.close()
```

L'esecuzione dello script fornisce il seguente risultato:

```
[(1, 'Italia'), (2, 'Gran Bretagna'), (3, 'USA'), (4, 'Russia')]
```

Se non ci piace questo modo grezzo di presentare il risultato e lo vogliamo esporre meglio possiamo intervenire inserendo, al posto dell'istruzione `print(risultato)`, il seguente blocchetto:

```
for i in range(len(risultato)):
    ri = risultato[i]
    r = str(ri[0]) + " " + ri[1]
    print r
```

⁴Sul mio computer, il database «biblioteca» si trova nella directory Database della mia home; essendo su un sistema Linux, come avverrebbe su un sistema Mac, l'indirizzo al database è indicato con le barre inclinate al contrario di quelle che si usano in Windows.

nel quale ogni elemento della lista risultato viene preso come una tupla isolata, ogni elemento della tupla viene accostato all'altro come stringa, con in mezzo un piccolo spazio libero, e ogni stringa risultante viene stampata.

Il risultato ora si presenta così:

```
1 Italia
2 Gran Bretagna
3 USA
4 Russia
```

Occorre ricordare che la stringa risultante da quest'ultima operazione è una stringa unicode. Pertanto, se essa, anziché sullo standard output, come avviene per default nel nostro esempio, fosse diretta a un file, la stringa `r` oggetto dell'istruzione `print` andrebbe scritta `r.encode('utf-8')`; in caso contrario avremmo un messaggio di errore e il file non potrebbe essere scritto.

* * *

L'approccio esemplificato utilizzando il linguaggio Python è praticamente lo stesso, magari con qualche complicazione formale in più, per tutti gli altri linguaggi di programmazione che ho prima ricordato.

Ci si collega a un database con una istruzione che, se il database non c'è, lo crea; si crea un cursore attraverso cui si eseguono comandi SQL per creare tabelle, alimentarle, leggerle. E tutto ciò possiamo inserirlo in un programma più ampio, vuoi per predisporre un ambiente grafico attorno al database come tale (ad esempio finestre per l'inserimento dei dati), vuoi per fornire un database ad un software destinato a tutt'altro scopo ma che abbia bisogno, al proprio interno, di una funzione di database.

6 Conversione di database

Ciascun database può essere salvato in un file in formato SQL.

Nel caso di database SQLite esiste un comando puntato

```
. dump
```

che lo fa se lo scriviamo dopo aver scritto un altro comando puntato

```
. output <nome_file>.sql
```

necessario per redirigere l'output che, per default, sarebbe allo schermo, su file.

Se, successivamente nella stessa sessione di lavoro, vogliamo redirigere l'output allo schermo usiamo il comando puntato `. output stdout`.

Il file prodotto con questo salvataggio, con estensione `.sql`, è un file di testo che contiene le istruzioni, in linguaggio SQL, per ricostruire, tale e quale, il database con le sue tabelle e i suoi dati.

Ciascun database ha anche modo di leggere un file in formato SQL.

Nel caso di database SQLite esiste un comando puntato

```
. read <nome_file>
```

che lo fa.

La lettura del file ha per risultato la ricostruzione del database da cui era stato originato il file `.sql`.

Purtroppo questi file in formato SQL sono degli ottimi strumenti di backup del database ma, contrariamente a quanto si potrebbe pensare, non si prestano, così come sono, ad operazioni di conversione.

Ciò soprattutto per colpa di alcune particolarità con cui, in SQLite, si designano i campi destinati ad ospitare le chiavi primarie.

Chi conosca tutti i segreti di queste particolarità, prima fra tutte quella, già citata nella nota 2 del Capitolo 3 relativa al fatto che `auto_increment` usato da MariaDB in stretta aderenza al linguaggio SQL in SQLite diventa `autoincrement`, potrebbe, con santa pazienza apportare le modifiche del caso nel file SQL, che è un semplice file di testo manipolabile con qualsiasi editor di testo.

Esistono software che fanno per noi questo lavoro, occupandosi di conversioni che non riguardano solo SQLite e MySQL ma anche altri tipi di database.

In proposito invito a diffidare di alcune cose miracolistiche reperibili navigando in rete e, con dolore, essendo un convinto sostenitore del software libero, devo segnalare le uniche cose serie che conosco, che sono software proprietari e costano parecchi soldi: RazorSQL, disponibile per Linux, Windows e Mac OS X e acquistabile all'indirizzo <https://razorsql.com/download.html> e DB Convert for SQLite & MySQL, disponibile solo per Windows (e ben funzionante su Wine per essere utilizzato anche in Linux e Mac OS X) e acquistabile all'indirizzo <https://dbconvert.com/sqlite/mysql/>.

Chi voglia risparmiare e sia disponibile a lavorare un pochino di più può comunque ricorrere all'uovo di Colombo: generare da zero le tabelle che abbiamo in un ambiente nell'altro, badando bene che i campi siano gli stessi e nello stesso ordine, e poi trasferirvi i dati con un file .csv per ciascuna tabella.

Il tutto, peraltro, è abbastanza semplice: le tabelle sappiamo già come generarle, rimane da vedere come generare e manipolare i file .csv e sapere come evitare alcune trappole. Prima trappola tra tutte, la scelta del carattere separatore nel file .csv, che è l'acronimo di comma separated values e, per default usa, appunto il simbolo comma, la virgola. Per chi, come me, ha il brutto vizio di usare la virgola nei database, per esempio come ho fatto negli esercizi di questo manualetto per separare il cognome degli autori dal loro nome, l'uso della virgola per separare i campi è un disastro: meglio il punto e virgola, che, nei database, penso nessuno usi.

Ciò premesso, vediamo come generare file .csv adatti per passare da SQLite a MySQL e viceversa.

6.1 da SQLite a MariaDB

Nel prompt di SQLite, con aperto il database su cui lavorare, scriviamo, nell'ordine, i seguenti comandi

```
. separator ';'
. output <nome_file>.csv
select * from <nome_tabella>
```

Il file .csv così ottenuto è importabile in phpMyAdmin, avendo cura di indicare il separatore di campo scelto (che, secondo quanto detto prima, ho scelto nel ;).

6.2 da MySQL a SQLite

Da phpMyAdmin esportiamo i dati della tabella nel formato Foglio di calcolo nel formato Open Document.

Apriamo il file così prodotto, che ha estensione .ods, con OpenOffice o LibreOffice Calc e formattiamo le celle contenenti date in modo che le date siano nel formato aaaa-mm-gg.

Salviamo il file come file di testo .csv, indicando come tipo carattere Unicode utf-8, come separatore di campo un carattere non utilizzato nel database, ad esempio, come ho detto prima, il punto e virgola e cancellando il separatore di testo in modo che la relativa finestra risulti vuota.

Il file .csv così ottenuto è ora importabile in SQLite, per riempire la tabella, con il comando

```
. import <nome_file>.csv <nome_tabella>.
```

Dal momento che il software DB Convert for SQLite & MySQL funziona come demo non registrata trasferendo al massimo 50 record per tabella, potremmo utilizzarlo come demo per convertire il nostro database.

In questo modo avremmo il database convertito, con tutte le tabelle costruite a dovere e contenenti tutti i relativi record, se meno di 50.

Per le tabelle contenenti più di 50 record potremmo cancellarne il contenuto limitato e riempirle con il sistema dei file .csv che ho appena descritto.

* * *

Il sistema di conversione attraverso la reimpostazione delle tabelle e il trasferimento dei dati che ho descritto per database SQLite e MariaDB è ovviamente applicabile, con le dovute varianti, alle conversioni tra SQLite e qualsiasi altro tipo di database che sia in grado di generare e di importare dati da file .csv.